

# Final Project Markdown

2025-12-08

#Libraries

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.6
## ✓ forcats   1.0.1      ✓ stringr   1.6.0
## ✓ ggplot2    4.0.1      ✓ tibble     3.3.0
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.2.0
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method           from
##   as.zoo.data.frame zoo
```

```
library(tseries)
library(rugarch)
```

```
## Loading required package: parallel
##
## Attaching package: 'rugarch'
##
## The following object is masked from 'package:purrr':
##
##   reduce
```

```
library(TSA)
```

```
## Registered S3 methods overwritten by 'TSA':
##   method      from
##   fitted.Arima forecast
##   plot.Arima   forecast
##
## Attaching package: 'TSA'
##
## The following object is masked from 'package:readr':
##
##     spec
##
## The following objects are masked from 'package:stats':
##
##     acf, arima
##
## The following object is masked from 'package:utils':
##
##     tar
```

```
library(tsDyn)
library(arfima)
```

```
## Loading required package: ltsa
## Note that the arfima package has new defaults starting with
## 1.4-0: type arfimachanges() for a list, as well as some other notes.
## NOTE: some of these are quite important!
##
## Attaching package: 'arfima'
##
## The following object is masked from 'package:forecast':
##
##     arfima
##
## The following object is masked from 'package:stats':
##
##     BIC
```

```
library(vars)
```

```
## Loading required package: MASS
##
## Attaching package: 'MASS'
##
## The following object is masked from 'package:dplyr':
##
##     select
##
## Loading required package: strucchange
## Loading required package: zoo
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
##
## Loading required package: sandwich
##
## Attaching package: 'strucchange'
##
## The following object is masked from 'package:stringr':
##
##     boundary
##
## Loading required package: urca
## Loading required package: lmtest
```

```
library(FinTS)
```

```
##
## Attaching package: 'FinTS'
##
## The following object is masked from 'package:forecast':
##
##     Acf
```

```
library(lmtest)
library(splines)
library(mgcv)
```

```
## Loading required package: nlme
##
## Attaching package: 'nlme'
##
## The following object is masked from 'package:forecast':
##
##     getResponse
##
## The following object is masked from 'package:dplyr':
##
##     collapse
##
## This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-10
```

# 1. IMPORT

```
bryson_raw <- read_csv("ASA All PGA Raw Data - Tourn Level.csv")
```

```
## Rows: 36864 Columns: 37
## — Column specification —————
## Delimiter: ","
## chr   (5): Player_initial_last, player, tournament name, course, Finish
## dbl  (28): tournament id, player id, hole_par, strokes, hole_DKP, hole_FDP, ...
## lgl   (3): Unnamed: 2, Unnamed: 3, Unnamed: 4
## date  (1): date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```

bryson_clean <- bryson_raw %>%
  filter(player == "Bryson DeChambeau") %>%
  filter(!is.na(pos)) %>%
  mutate(date = as.Date(date)) %>%
  arrange(date)

bryson_tourn <- bryson_clean %>%
  group_by(`tournament id`) %>%
  summarise(
    date      = first(date),
    pos       = first(pos),
    sg_putt   = mean(sg_putt, na.rm = TRUE),
    sg_ott    = mean(sg_ott, na.rm = TRUE),
    sg_app    = mean(sg_app, na.rm = TRUE),
    sg_arg    = mean(sg_arg, na.rm = TRUE),
    sg_total  = mean(sg_total, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(date)

head(bryson_tourn)

```

```

## # A tibble: 6 × 8
##   `tournament id` date      pos sg_putt sg_ott sg_app sg_arg sg_total
##           <dbl> <date>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1           2250 2015-06-14    45    0.31    0.29    0.72   -0.51    0.81
## 2           2726 2016-03-16     2   NaN     NaN     NaN     NaN     NaN
## 3           2493 2016-04-10    21   NaN     NaN     NaN     NaN     NaN
## 4           2494 2016-04-17     4  -0.01    0.25    1.38    1.02    2.65
## 5           2499 2016-06-05    38  -0.74    0.12   -0.02    1.43    0.79
## 6           2501 2016-06-19    15   NaN     NaN     NaN     NaN     NaN

```

## 2. BOX-COX TRANSFORMATION

```

# Remove tournaments with missing SG putting
sg_putt_clean <- bryson_tourn %>%
  drop_na(sg_putt) %>%
  pull(sg_putt)

# Convert to ts
sg_putt_ts <- ts(sg_putt_clean, frequency = 1)

sg_pos <- sg_putt_ts - min(sg_putt_ts) + 1

lambda <- BoxCox.lambda(sg_pos)
lambda

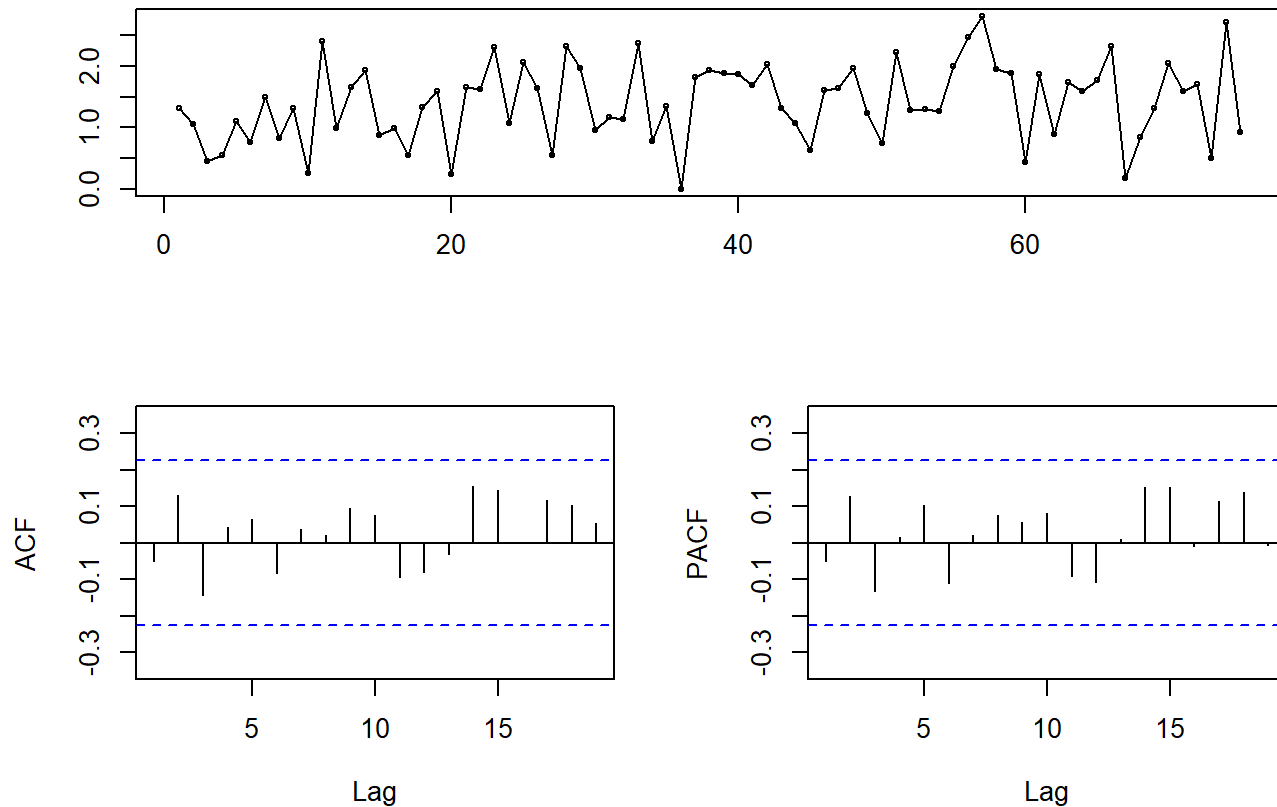
```

```
## [1] 0.7004776
```

```
sg_putt_trans <- BoxCox(sg_pos, lambda)
sg_putt_ts_trans <- ts(sg_putt_trans, frequency = 1)

tsdisplay(sg_putt_ts_trans, main = "Transformed SG Putting (Tournament-Level)")
```

### Transformed SG Putting (Tournament-Level)



## 3. STATIONARITY TESTS

```
adf.test(sg_putt_ts_trans)
```

```
##
## Augmented Dickey-Fuller Test
##
## data: sg_putt_ts_trans
## Dickey-Fuller = -3.9249, Lag order = 4, p-value = 0.01781
## alternative hypothesis: stationary
```

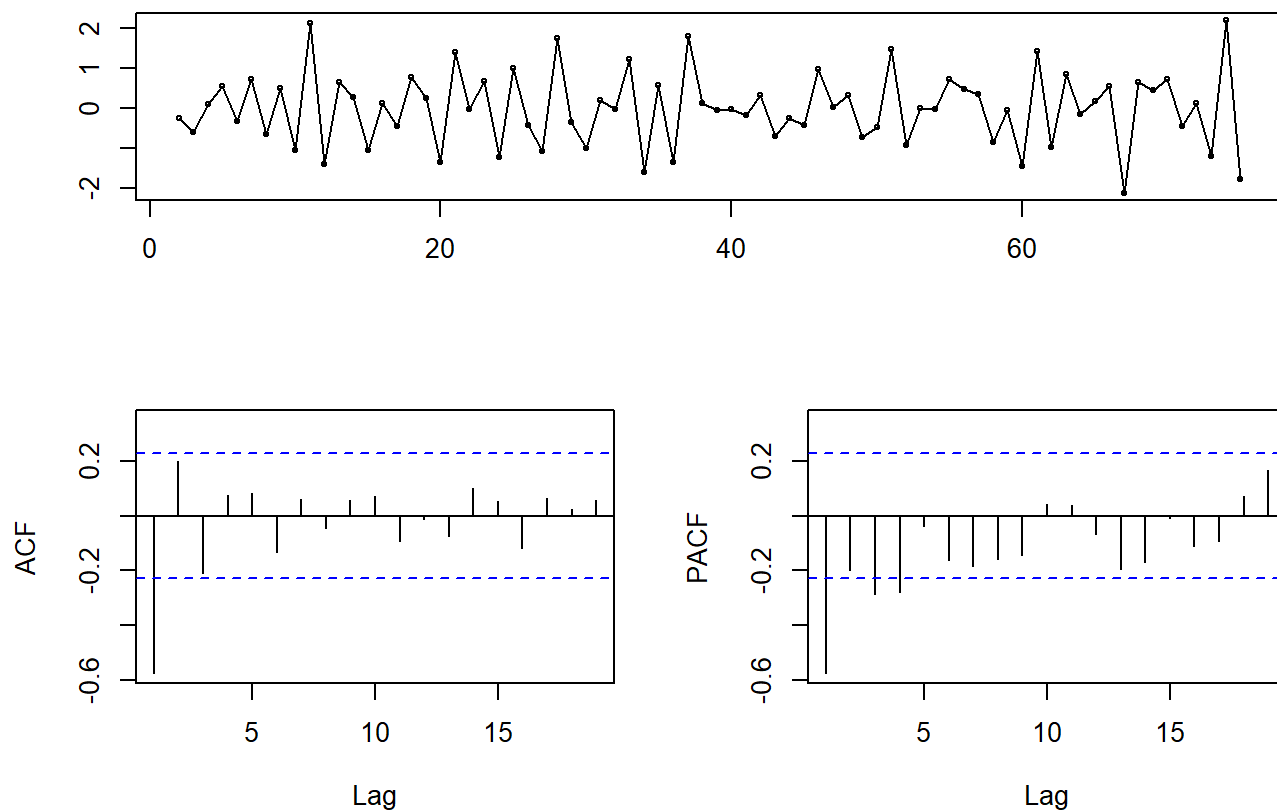
```
kpss.test(sg_putt_ts_trans)
```

```
##
## KPSS Test for Level Stationarity
##
## data: sg_putt_ts_trans
## KPSS Level = 0.4702, Truncation lag parameter = 3, p-value = 0.04838
```

## 4. DIFFERENCING

```
sg_diff1 <- diff(sg_putt_ts_trans)
tsdisplay(sg_diff1, main = "First Difference of SG Putting")
```

**First Difference of SG Putting**



```
adf.test(sg_diff1)
```

```
## Warning in adf.test(sg_diff1): p-value smaller than printed p-value
```

```
##
## Augmented Dickey-Fuller Test
##
## data: sg_diff1
## Dickey-Fuller = -5.8407, Lag order = 4, p-value = 0.01
## alternative hypothesis: stationary
```

```
kpss.test(sg_diff1)
```

```
## Warning in kpss.test(sg_diff1): p-value greater than printed p-value
```

```
##
## KPSS Test for Level Stationarity
##
## data: sg_diff1
## KPSS Level = 0.036888, Truncation lag parameter = 3, p-value = 0.1
```

```
returns_ts <- na.omit(ts(as.numeric(sg_diff1)))
```

## 5. BASIC FORECAST MODELS

```
naive_fc <- naive(sg_diff1, h = 5)
mean_fc <- meanf(sg_diff1, h = 5)
drift_fc <- rwf(sg_putt_ts_trans, drift = TRUE, h = 5)

naive_model <- Arima(sg_diff1, order = c(0,0,0), include.mean = FALSE)
mean_model <- Arima(sg_diff1, order = c(0,0,0), include.mean = TRUE)
drift_model <- Arima(sg_putt_ts_trans, order = c(0,0,0), include.drift = TRUE)

data.frame(
  Model = c("Naive", "Mean", "Drift"),
  AIC = c(
    AIC(naive_model),
    AIC(mean_model),
    AIC(drift_model)
  )
)
```

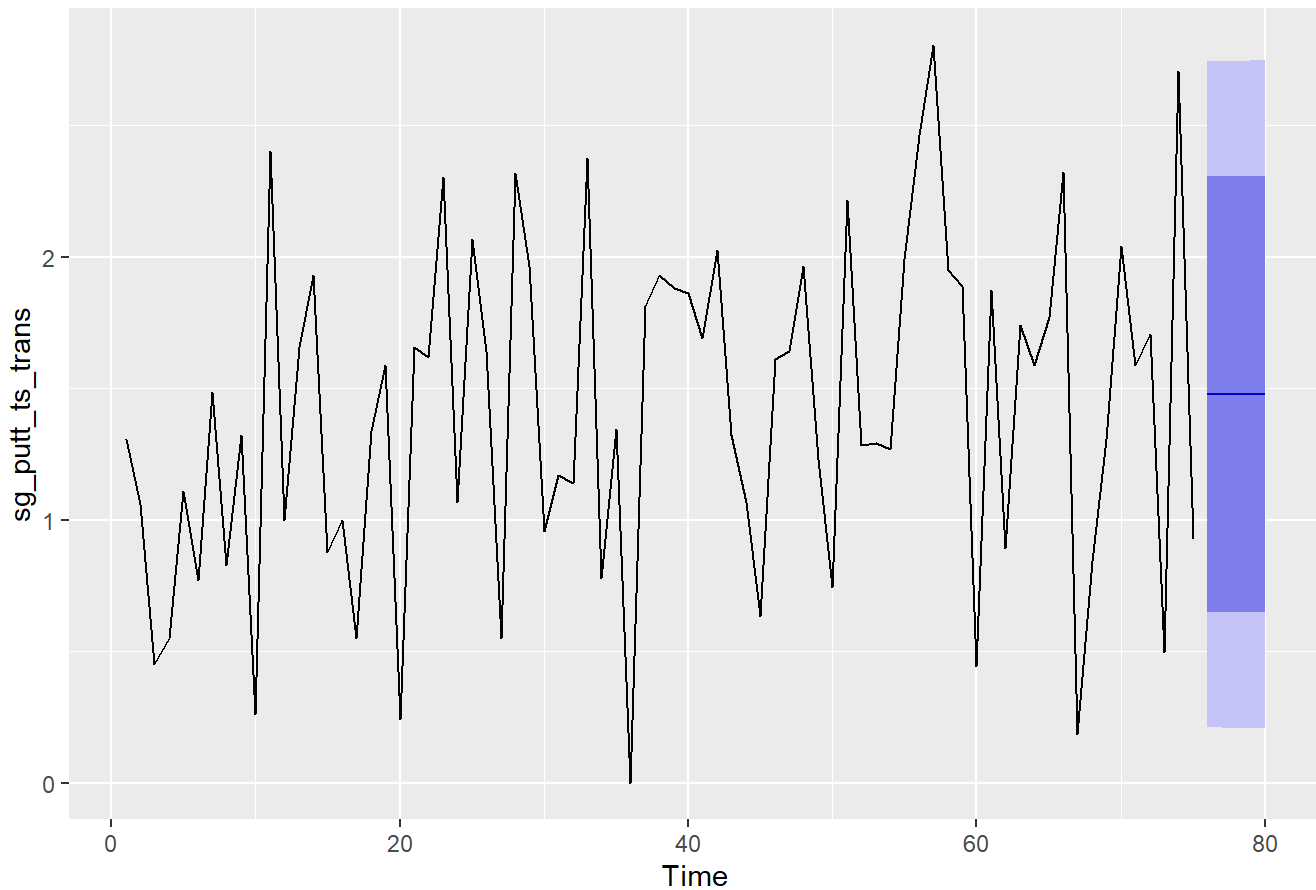
```
## Model AIC
## 1 Naive 200.4445
## 2 Mean 202.4422
## 3 Drift 146.6062
```

## 6. EXPONENTIAL SMOOTHING

```
ets_model <- ets(sg_putt_ts_trans)
ets_fc <- forecast(ets_model, h = 5)
autoplot(ets_fc)
```



## Forecasts from ETS(A,N,N)



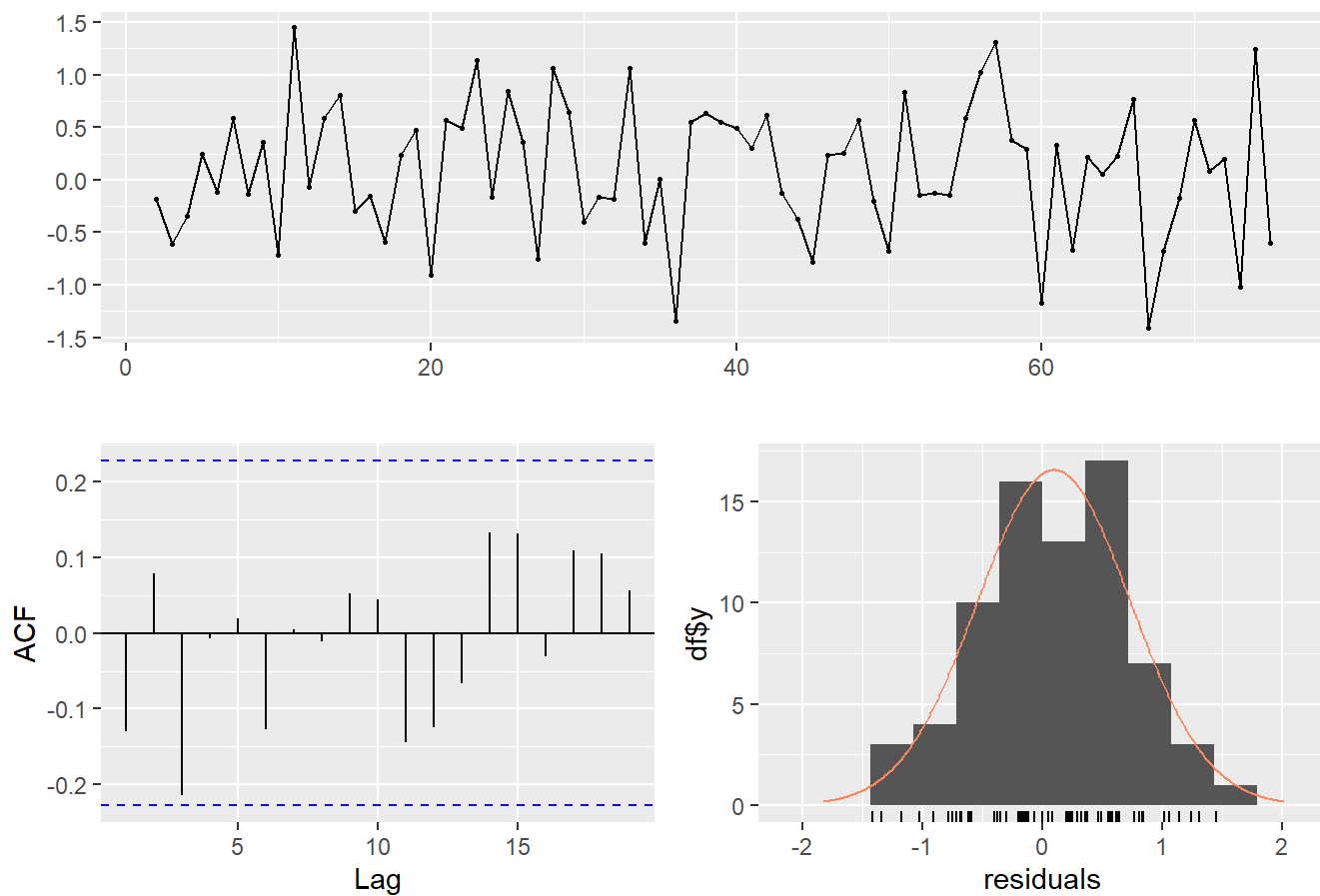
## 7. SARIMA MODEL

```
sarima_model <- auto.arima(  
  sg_diff1,  
  seasonal = FALSE,  
  stepwise = FALSE,  
  approximation = FALSE  
)  
summary(sarima_model)
```

```
## Series: sg_diff1
## ARIMA(0,0,1) with zero mean
##
## Coefficients:
##          ma1
##        -0.9427
## s.e.    0.0427
##
## sigma^2 = 0.4183: log likelihood = -73.35
## AIC=150.7   AICc=150.87   BIC=155.31
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.09777299 0.6424094 0.5304461 -12.37601 211.2533 0.4030001
##              ACF1
## Training set -0.1295526
```

```
checkresiduals(sarima_model)
```

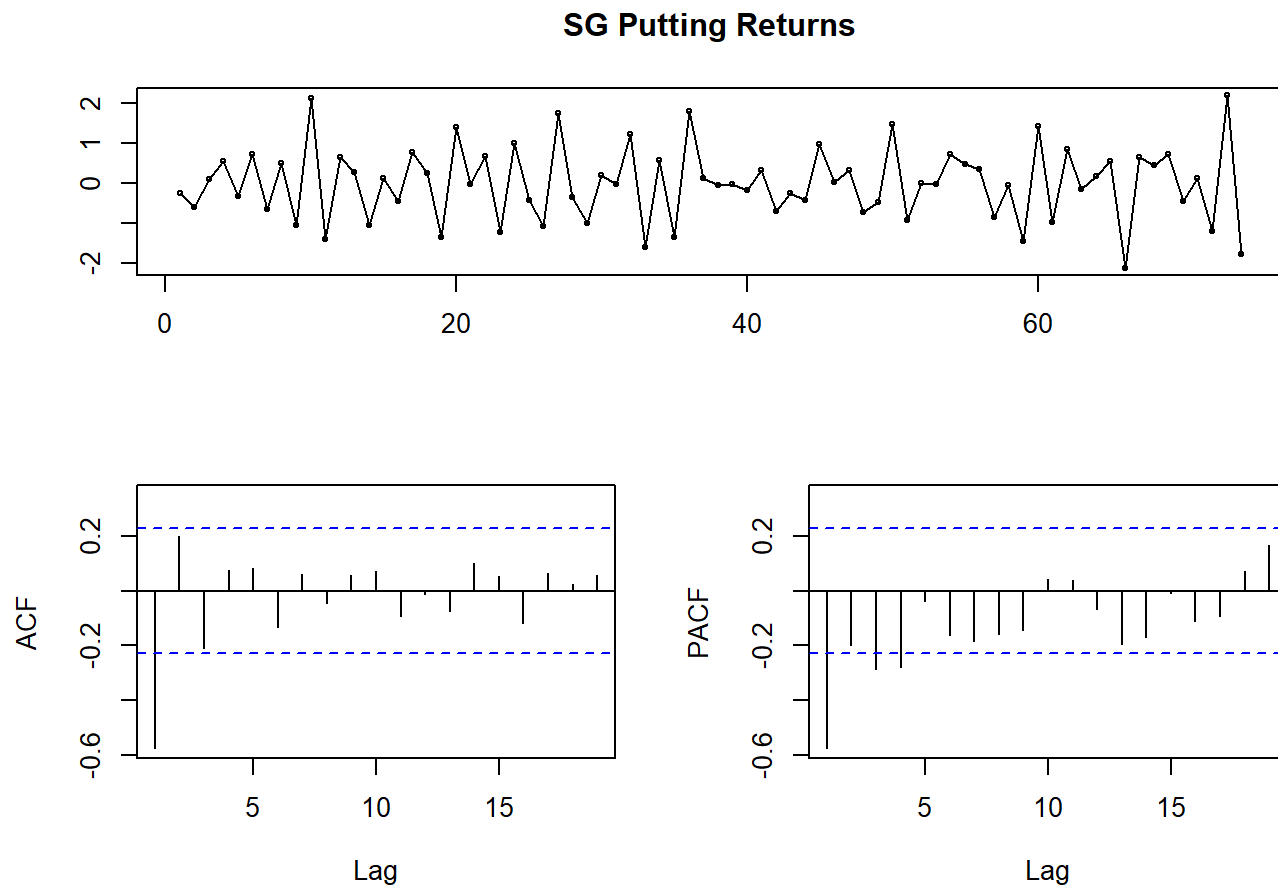
Residuals from ARIMA(0,0,1) with zero mean



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,0,1) with zero mean
## Q* = 7.2012, df = 9, p-value = 0.6162
##
## Model df: 1.   Total lags used: 10
```

## 8. ARCH GARCH

```
tsdisplay(returns_ts, main = "SG Putting Returns")
```



```
ArchTest(returns_ts, lags = 10)
```

```
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  returns_ts
## Chi-squared = 18.452, df = 10, p-value = 0.04781
```

```
garch11_spec <- ugarchspec(
  variance.model=list(model="sGARCH", garchOrder=c(1,1)),
  mean.model=list(armaOrder=c(0,0)),
  distribution.model="norm"
)
garch11_fit <- ugarchfit(spec=garch11_spec, data=returns_ts)
```

```
## Warning in .sgarchfit(spec = spec, data = data, out.sample = out.sample, :
## ugarchfit-->waring: using less than 100 data
## points for estimation
```

*# t-distribution*

```
garch11_t_spec <- ugarchspec(
  variance.model=list(model="sGARCH", garchOrder=c(1,1)),
  mean.model=list(armaOrder=c(0,0)),
  distribution.model="std"
)
garch11_t_fit <- ugarchfit(spec=garch11_t_spec, data=returns_ts)
```

```
## Warning in .sgarchfit(spec = spec, data = data, out.sample = out.sample, :
## ugarchfit-->waring: using less than 100 data
## points for estimation
```

*# EGARCH*

```
egarch11_spec <- ugarchspec(
  variance.model=list(model="eGARCH", garchOrder=c(1,1)),
  mean.model=list(armaOrder=c(0,0)),
  distribution.model="std"
)
egarch11_fit <- ugarchfit(spec=egarch11_spec, data=returns_ts)
```

```
## Warning in .egarchfit(spec = spec, data = data, out.sample = out.sample, :
## ugarchfit-->waring: using less than 100 data
## points for estimation
```

## 9. TAR MODELS

```
Tsay.test(returns_ts)
```

```
## $test.stat
## [1] 0.3796
##
## $p.value
## [1] 0.9504
##
## $order
## [1] 4
```

```
tar_m2    <- setar(returns_ts, m=2, thDelay=0)
tar_m3    <- setar(returns_ts, m=3, thDelay=0)
```

```
## Warning: Possible unit root in the low regime. Roots are: 0.9534 0.9534 2.238
```

```
tar_m2_d1 <- setar(returns_ts, m=2, thDelay=1)
```

## 10. FARIMA

```
library(arfima)
```

```
farima_model <- arfima(returns_ts)
```

```
## Note: only one starting point. Only one mode can be found -- this is now the default behavior.
## Beginning the fits with 1 starting values.
```

```
summary(farima_model)
```

```
##
## Call:
##
## arfima(z = returns_ts)
##
##
## Mode 1 Coefficients:
##              Estimate   Std. Error Th. Std. Err.      z-value Pr(>|z|)
## d.f            -9.99275e-01  6.32456e-07   9.06366e-02 -1.57999e+06 < 2e-16 ***
## Fitted mean   6.71226e-03  3.32866e-03           NA  2.01650e+00 0.043747 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## sigma^2 estimated as 0.397694; Log-likelihood = 32.9745; AIC = -59.949; BIC = -53.0368
##
## Numerical Correlations of Coefficients:
##              d.f  Fitted mean
## d.f              1.00 0.00
## Fitted mean 0.00 1.00
##
## Theoretical Correlations of Coefficients:
##      d.f
## d.f 1.00
##
## Expected Fisher Information Matrix of Coefficients:
##      d.f
## d.f 1.64
```

```
d_hat <- coef(farima_model)[1]
d_hat
```

```
## [1] -0.9992751
```

## 11. MODEL COMPARISON BY AIC

```

aic_table <- tibble(
  Model = c("Naive", "Mean", "Drift", "ETS", "SARIMA", "GARCH11", "GARCH11-t", "EGARCH", "TAR2", "TAR3", "TA
R2d1", "FARIMA"),
  AIC = c(
    AIC(naive_model),
    AIC(mean_model),
    AIC(drift_model),
    AIC(ets_model),
    AIC(sarima_model),
    infocriteria(garch11_fit)[1],
    infocriteria(garch11_t_fit)[1],
    infocriteria(egarch11_fit)[1],
    AIC(tar_m2),
    AIC(tar_m3),
    AIC(tar_m2_d1),
    AIC(farima_model)
  )
)

aic_table %>% arrange(AIC)

```

```

## # A tibble: 12 × 2
##   Model      AIC
##   <chr>    <dbl>
## 1 FARIMA  -59.9
## 2 TAR3    -46.7
## 3 TAR2    -42.9
## 4 TAR2d1  -37.6
## 5 EGARCH   2.73
## 6 GARCH11  2.79
## 7 GARCH11-t 2.82
## 8 Drift    147.
## 9 SARIMA   151.
## 10 Naive   200.
## 11 Mean    202.
## 12 ETS     262.

```

## 12. ARIMAX

```

min_len <- min(length(sg_diff1), length(bryson_tourn$sg_total))

y <- sg_diff1[1:min_len]
x <- bryson_tourn$sg_total[1:min_len]

arimax_model <- auto.arima(
y,
xreg = x,
stepwise = FALSE,
approximation = FALSE
)

summary(arimax_model)

```

```

## Series: y
## Regression with ARIMA(4,0,0) errors
##
## Coefficients:
##          ar1      ar2      ar3      ar4      xreg
##      -0.7515  -0.3917  -0.4843  -0.2788  0.0106
## s.e.    0.1335   0.1617   0.1505   0.1305  0.0244
##
## sigma^2 = 0.375:  log likelihood = -64.22
## AIC=140.43   AICc=141.69   BIC=154.26
##
## Training set error measures:
##              ME      RMSE      MAE      MPE      MAPE      MASE
## Training set 0.04609559 0.6567168 0.5367246 -33.72607 250.8583 0.4077701
##              ACF1
## Training set -0.03868529

```

```
coeftest(arimax_model)
```

```

##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## ar1  -0.751527   0.133525 -5.6284 1.819e-08 ***
## ar2  -0.391702   0.161686 -2.4226 0.015409 *
## ar3  -0.484322   0.150531 -3.2174 0.001294 **
## ar4  -0.278775   0.130548 -2.1354 0.032727 *
## xreg  0.010573   0.024350  0.4342 0.664129
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

## ARIMAX (SG PUTTING IN LEVELS, NO MANUAL DIFFERENCING)



```

min_len <- min(length(bryson_tourn$sg_putt),
               length(bryson_tourn$sg_total))

y <- bryson_tourn$sg_putt[1:min_len]
x <- bryson_tourn$sg_total[1:min_len]

arimax_levels <- auto.arima(
  y,
  xreg = x,
  stepwise = FALSE,
  approximation = FALSE
)

summary(arimax_levels)

```

```

## Series: y
## Regression with ARIMA(0,0,0) errors
##
## Coefficients:
##          xreg
##          0.3145
## s.e.  0.0385
##
## sigma^2 = 0.5097:  log likelihood = -80.65
## AIC=165.29  AICc=165.43  BIC=170.34
##
## Training set error measures:
##              ME      RMSE      MAE  MPE MAPE      MASE      ACF1
## Training set -0.001995643 0.7091706 0.5836347 -Inf  Inf 0.5765017 -0.0296215

```

```
coeftest(arimax_levels)
```

```

##
## z test of coefficients:
##
##      Estimate Std. Error z value Pr(>|z|)
## xreg 0.314506   0.038483  8.1727 3.015e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

## 13. VAR MODEL

```
var_df <- bryson_tourn %>%  
  dplyr::select(pos, sg_putt, sg_ott, sg_app, sg_arg) %>%  
  drop_na()  
  
var_ts <- ts(var_df)  
  
var_sel <- VARselect(var_ts, lag.max = 6, type = "const")  
  
var_model <- VAR(var_ts, p = var_sel$selection["AIC(n)"], type = "const")  
summary(var_model)
```

```
##
## VAR Estimation Results:
## =====
## Endogenous variables: pos, sg_putt, sg_ott, sg_app, sg_arg
## Deterministic variables: const
## Sample size: 74
## Log Likelihood: -563.459
## Roots of the characteristic polynomial:
## 0.4271 0.2187 0.2014 0.2014 0.05585
## Call:
## VAR(y = var_ts, p = var_sel$selection["AIC(n)"], type = "const")
##
##
## Estimation results for equation pos:
## =====
## pos = pos.l1 + sg_putt.l1 + sg_ott.l1 + sg_app.l1 + sg_arg.l1 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## pos.l1      0.2683      0.2158   1.244  0.2179
## sg_putt.l1   4.2363      3.3585   1.261  0.2115
## sg_ott.l1   -1.3945      4.4427  -0.314  0.7546
## sg_app.l1    0.2757      3.4342   0.080  0.9363
## sg_arg.l1    8.7624      4.0766   2.149  0.0352 *
## const      13.2104      8.8890   1.486  0.1419
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 17.63 on 68 degrees of freedom
## Multiple R-Squared: 0.1184, Adjusted R-squared: 0.05358
## F-statistic: 1.827 on 5 and 68 DF, p-value: 0.1192
##
##
## Estimation results for equation sg_putt:
## =====
## sg_putt = pos.l1 + sg_putt.l1 + sg_ott.l1 + sg_app.l1 + sg_arg.l1 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## pos.l1     -0.01058    0.01021  -1.036  0.3037
## sg_putt.l1 -0.18649    0.15886  -1.174  0.2445
## sg_ott.l1   0.11177    0.21014   0.532  0.5965
## sg_app.l1   0.04992    0.16244   0.307  0.7595
## sg_arg.l1  -0.34449    0.19282  -1.787  0.0785 .
## const      0.72107    0.42046   1.715  0.0909 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.8338 on 68 degrees of freedom
## Multiple R-Squared: 0.1176, Adjusted R-squared: 0.05271
## F-statistic: 1.812 on 5 and 68 DF, p-value: 0.122
##
```

```
##
## Estimation results for equation sg_ott:
## =====
## sg_ott = pos.l1 + sg_putt.l1 + sg_ott.l1 + sg_app.l1 + sg_arg.l1 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## pos.l1      -0.009705   0.007033  -1.380  0.17215
## sg_putt.l1  -0.049728   0.109474  -0.454  0.65110
## sg_ott.l1    0.189712   0.144814   1.310  0.19459
## sg_app.l1   -0.250096   0.111942  -2.234  0.02876 *
## sg_arg.l1   -0.028878   0.132879  -0.217  0.82860
## const       0.888943   0.289745   3.068  0.00309 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.5746 on 68 degrees of freedom
## Multiple R-Squared: 0.1316, Adjusted R-squared: 0.06777
## F-statistic: 2.061 on 5 and 68 DF, p-value: 0.08097
##
##
## Estimation results for equation sg_app:
## =====
## sg_app = pos.l1 + sg_putt.l1 + sg_ott.l1 + sg_app.l1 + sg_arg.l1 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## pos.l1      -0.01277    0.01019  -1.252  0.2148
## sg_putt.l1  -0.30039    0.15868  -1.893  0.0626 .
## sg_ott.l1   -0.27924    0.20991  -1.330  0.1879
## sg_app.l1   -0.09867    0.16226  -0.608  0.5451
## sg_arg.l1   -0.28490    0.19261  -1.479  0.1437
## const       0.97414    0.41998   2.319  0.0234 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.8329 on 68 degrees of freedom
## Multiple R-Squared: 0.06562, Adjusted R-squared: -0.003084
## F-statistic: 0.9551 on 5 and 68 DF, p-value: 0.4515
##
##
## Estimation results for equation sg_arg:
## =====
## sg_arg = pos.l1 + sg_putt.l1 + sg_ott.l1 + sg_app.l1 + sg_arg.l1 + const
##
##           Estimate Std. Error t value Pr(>|t|)
## pos.l1       0.001062   0.007293   0.146   0.885
## sg_putt.l1  -0.127330   0.113528  -1.122   0.266
## sg_ott.l1   -0.162543   0.150176  -1.082   0.283
## sg_app.l1   -0.010351   0.116087  -0.089   0.929
## sg_arg.l1    0.027796   0.137799   0.202   0.841
## const      0.200177    0.300474   0.666   0.508
```

```
##
##
## Residual standard error: 0.5959 on 68 degrees of freedom
## Multiple R-Squared: 0.08127, Adjusted R-squared: 0.01372
## F-statistic: 1.203 on 5 and 68 DF, p-value: 0.3171
##
##
##
## Covariance matrix of residuals:
##          pos  sg_putt  sg_ott  sg_app  sg_arg
## pos      310.732 -5.78556 -4.79303 -8.63481 -2.23937
## sg_putt   -5.786  0.69522  0.02730 -0.04851 -0.04850
## sg_ott    -4.793  0.02730  0.33015  0.13591 -0.07339
## sg_app    -8.635 -0.04851  0.13591  0.69366  0.01836
## sg_arg    -2.239 -0.04850 -0.07339  0.01836  0.35505
##
## Correlation matrix of residuals:
##          pos  sg_putt  sg_ott  sg_app  sg_arg
## pos      1.0000 -0.39363 -0.47322 -0.58815 -0.21320
## sg_putt  -0.3936  1.00000  0.05697 -0.06986 -0.09763
## sg_ott   -0.4732  0.05697  1.00000  0.28401 -0.21435
## sg_app   -0.5881 -0.06986  0.28401  1.00000  0.03700
## sg_arg   -0.2132 -0.09763 -0.21435  0.03700  1.00000
```

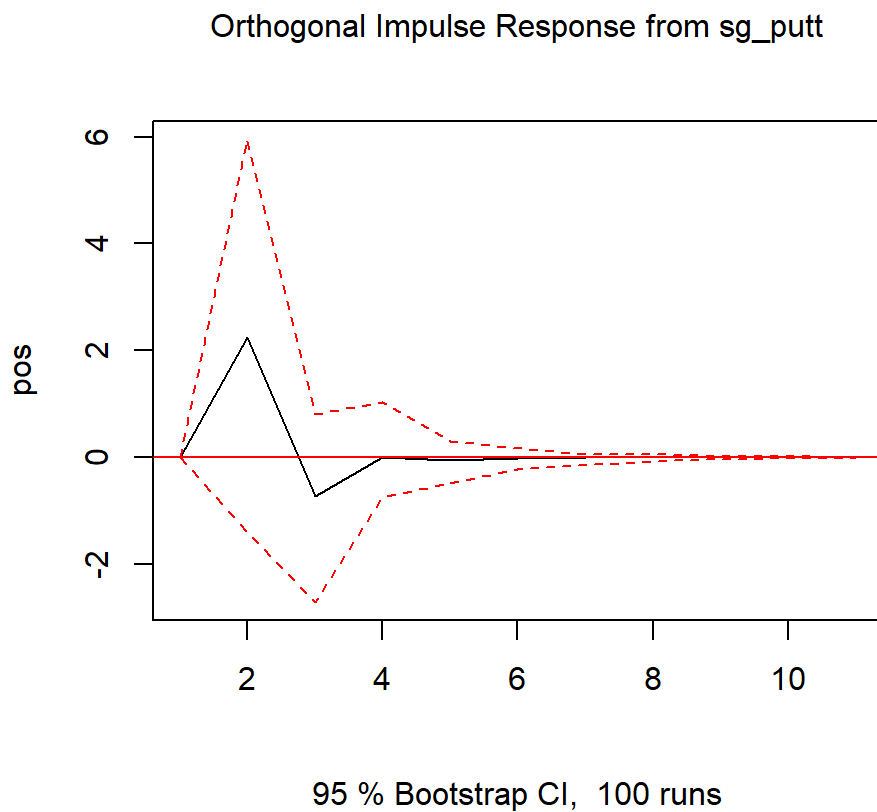
```
causality(var_model, cause="sg_putt")$Granger
```

```
##
## Granger causality H0: sg_putt do not Granger-cause pos sg_ott sg_app
## sg_arg
##
## data: VAR object var_model
## F-Test = 1.1865, df1 = 4, df2 = 340, p-value = 0.3165
```

```
causality(var_model, cause="sg_app")$Granger
```

```
##
## Granger causality H0: sg_app do not Granger-cause pos sg_putt sg_ott
## sg_arg
##
## data: VAR object var_model
## F-Test = 1.8662, df1 = 4, df2 = 340, p-value = 0.1159
```

```
irf_putt_on_pos <- irf(var_model, impulse="sg_putt", response="pos", n.ahead=10)
plot(irf_putt_on_pos)
```



## 14. REGRESSION

```
reg_df <- bryson_tourn %>%  
  dplyr::select(pos, sg_putt, sg_ott, sg_app, sg_arg, sg_total) %>%  
  drop_na()  
  
# Linear model  
reg1 <- lm(pos ~ sg_putt + sg_ott + sg_app + sg_arg, data = reg_df)  
summary(reg1)
```

```
##
## Call:
## lm(formula = pos ~ sg_putt + sg_ott + sg_app + sg_arg, data = reg_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -29.442  -5.401  -1.188   6.405  21.191
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   37.615      1.992  18.886 < 2e-16 ***
## sg_putt      -10.263      1.414   -7.259 4.20e-10 ***
## sg_ott       -11.775      2.145   -5.489 6.10e-07 ***
## sg_app       -10.178      1.493   -6.815 2.71e-09 ***
## sg_arg        -8.678      2.077   -4.178 8.36e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.24 on 70 degrees of freedom
## Multiple R-squared:  0.7012, Adjusted R-squared:  0.6841
## F-statistic: 41.06 on 4 and 70 DF,  p-value: < 2.2e-16
```

```
# Polynomial
reg_poly <- lm(
  pos ~ poly(sg_putt,2) + poly(sg_ott,2) + poly(sg_app,2) + poly(sg_arg,2),
  data = reg_df
)
summary(reg_poly)
```

```
##
## Call:
## lm(formula = pos ~ poly(sg_putt, 2) + poly(sg_ott, 2) + poly(sg_app,
##      2) + poly(sg_arg, 2), data = reg_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -29.6374  -4.7165  -0.8861   4.6715  21.4574
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      20.720      1.143   18.135 < 2e-16 ***
## poly(sg_putt, 2)1 -78.925     10.361   -7.618 1.28e-10 ***
## poly(sg_putt, 2)2   5.663     10.718    0.528  0.5990
## poly(sg_ott, 2)1  -59.534     11.519   -5.169 2.37e-06 ***
## poly(sg_ott, 2)2   24.438     10.448    2.339  0.0224 *
## poly(sg_app, 2)1  -68.846     11.191   -6.152 5.04e-08 ***
## poly(sg_app, 2)2    5.834     10.441    0.559  0.5782
## poly(sg_arg, 2)1  -47.925     10.581   -4.530 2.54e-05 ***
## poly(sg_arg, 2)2   12.953     10.707    1.210  0.2307
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.895 on 66 degrees of freedom
## Multiple R-squared:  0.737, Adjusted R-squared:  0.7051
## F-statistic: 23.11 on 8 and 66 DF, p-value: < 2.2e-16
```

```
# Splines
library(splines)
reg_spline <- lm(
  pos ~ bs(sg_putt,4) + bs(sg_ott,4) + bs(sg_app,4) + bs(sg_arg,4),
  data = reg_df
)
summary(reg_spline)
```

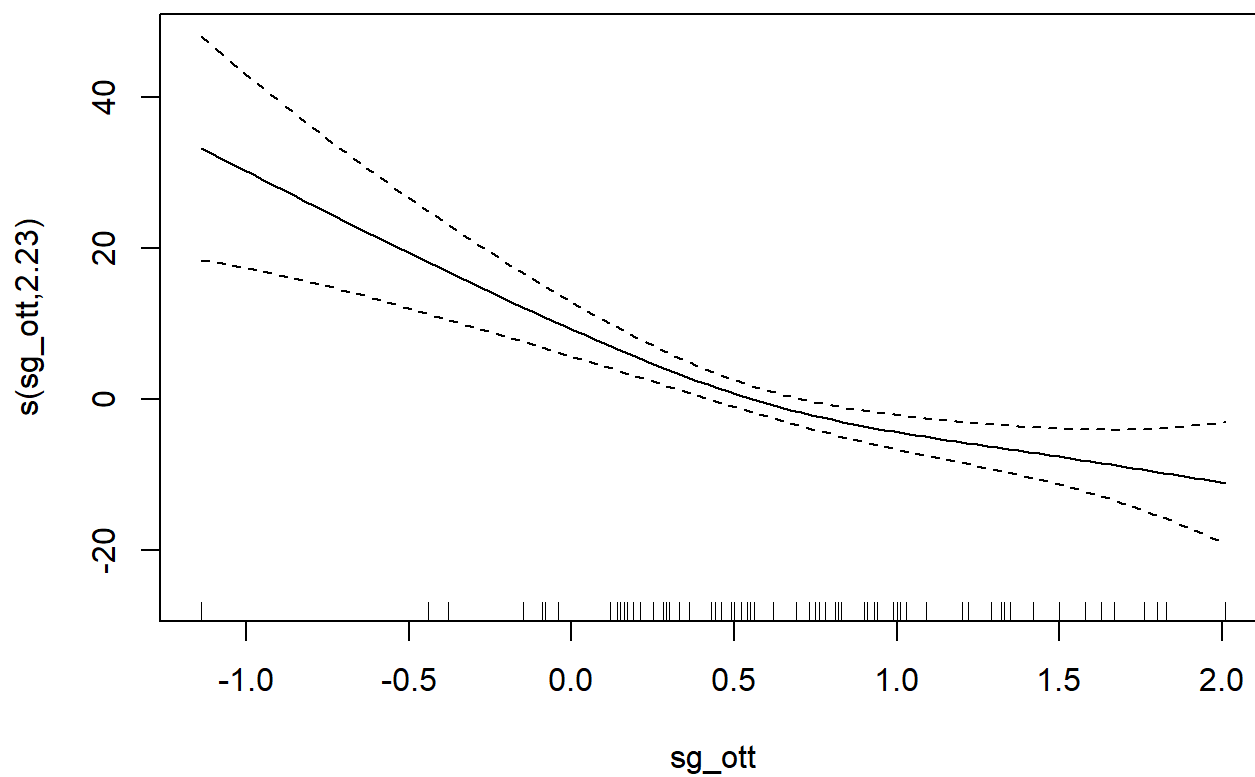
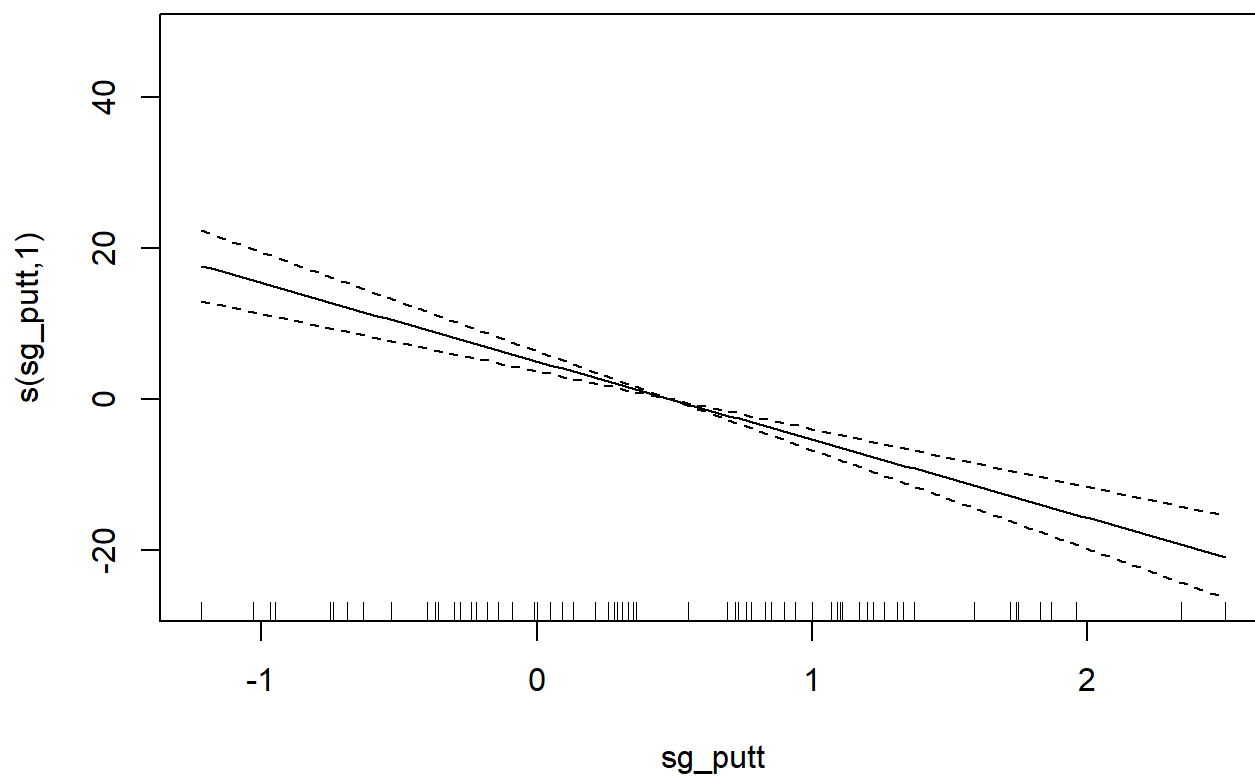


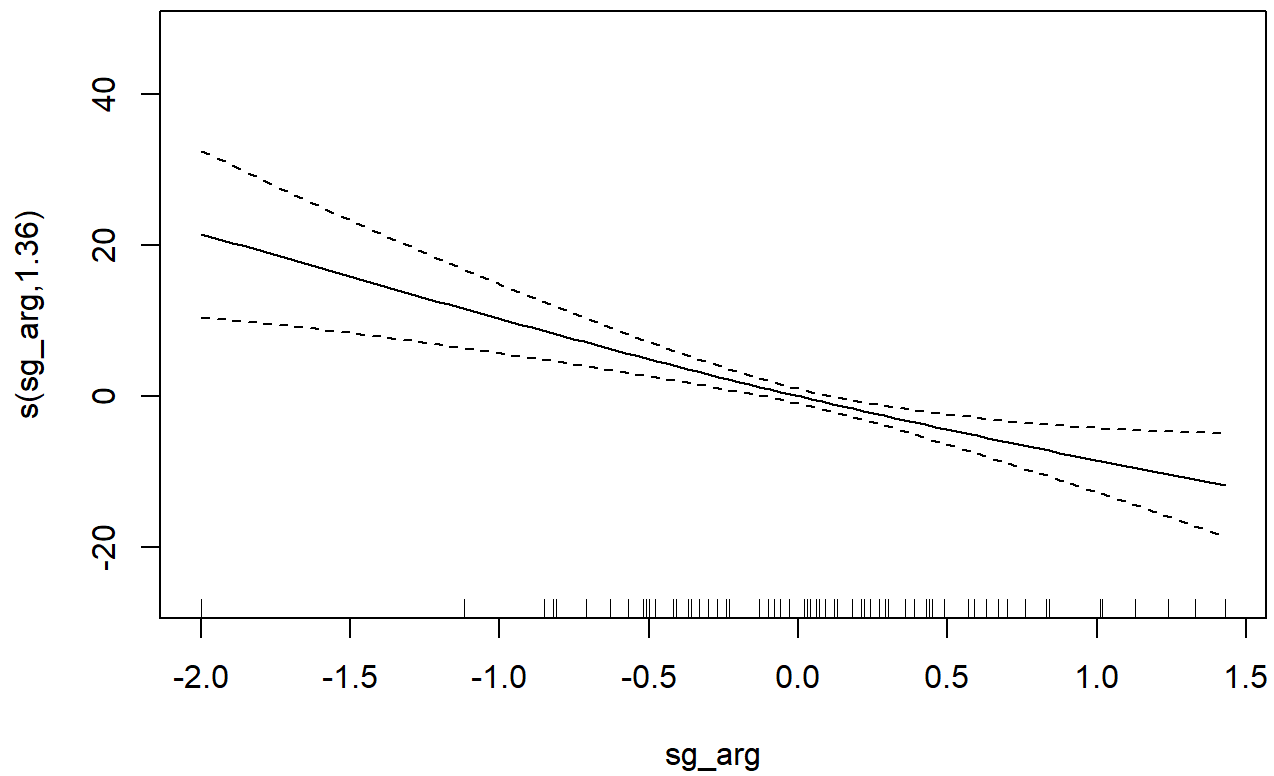
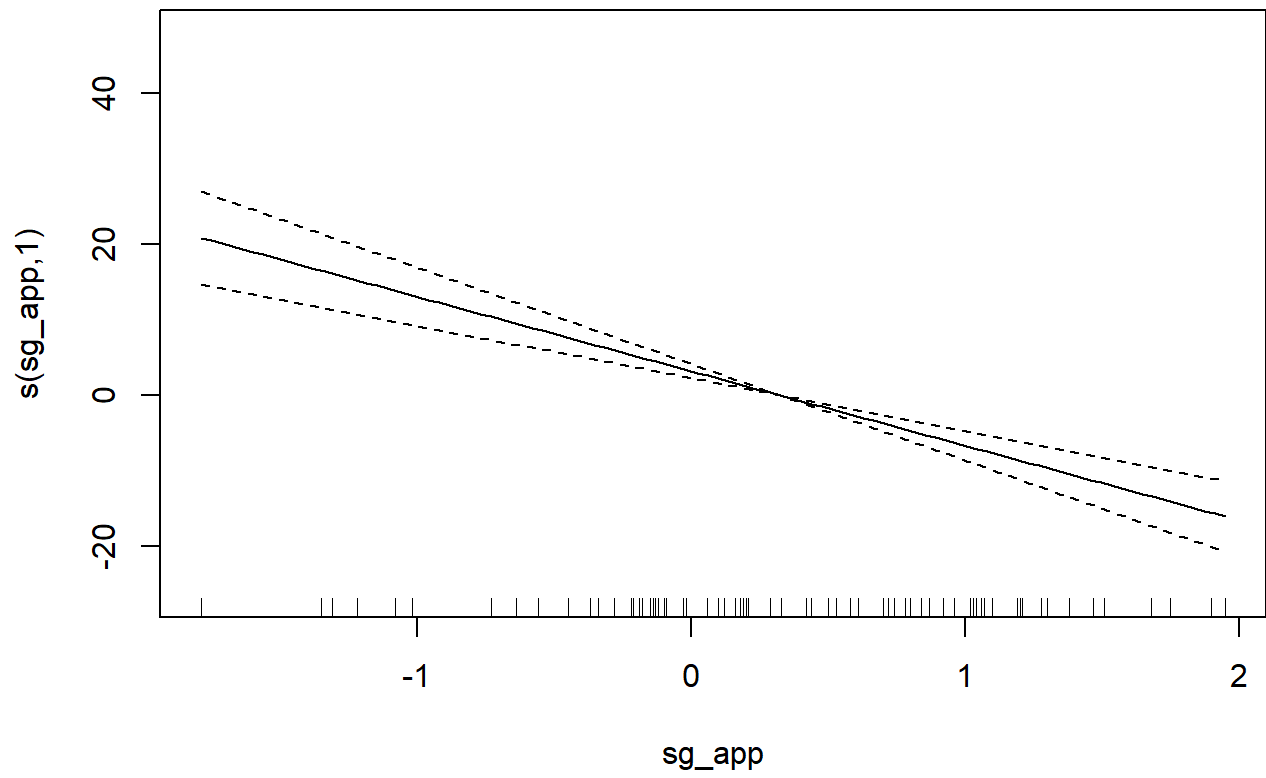
```
##
## Call:
## lm(formula = pos ~ bs(sg_putt, 4) + bs(sg_ott, 4) + bs(sg_app,
##      4) + bs(sg_arg, 4), data = reg_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -26.4861  -4.6491  -0.6925   4.7651  21.5256
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      145.70      23.56   6.185 6.75e-08 ***
## bs(sg_putt, 4)1    -20.01      14.32  -1.397 0.167680
## bs(sg_putt, 4)2    -22.95      11.39  -2.015 0.048571 *
## bs(sg_putt, 4)3    -40.78      15.71  -2.596 0.011921 *
## bs(sg_putt, 4)4   -48.02      14.43  -3.328 0.001522 **
## bs(sg_ott, 4)1     -3.06      19.10  -0.160 0.873248
## bs(sg_ott, 4)2    -54.32      15.31  -3.548 0.000777 ***
## bs(sg_ott, 4)3    -34.56      16.37  -2.111 0.039068 *
## bs(sg_ott, 4)4    -50.03      14.22  -3.518 0.000853 ***
## bs(sg_app, 4)1    -33.93      24.18  -1.404 0.165794
## bs(sg_app, 4)2    -17.05      11.13  -1.532 0.130893
## bs(sg_app, 4)3    -52.56      18.40  -2.856 0.005936 **
## bs(sg_app, 4)4    -42.70      14.03  -3.044 0.003502 **
## bs(sg_arg, 4)1    -25.57      19.07  -1.340 0.185338
## bs(sg_arg, 4)2    -21.48      15.71  -1.367 0.177009
## bs(sg_arg, 4)3    -42.89      13.65  -3.142 0.002643 **
## bs(sg_arg, 4)4    -36.40      13.64  -2.669 0.009861 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.03 on 58 degrees of freedom
## Multiple R-squared:  0.7623, Adjusted R-squared:  0.6967
## F-statistic: 11.63 on 16 and 58 DF,  p-value: 1.08e-12
```

```
# GAM
library(mgcv)
reg_gam <- gam(pos ~ s(sg_putt) + s(sg_ott) + s(sg_app) + s(sg_arg),
               data = reg_df)
summary(reg_gam)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## pos ~ s(sg_putt) + s(sg_ott) + s(sg_app) + s(sg_arg)
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  20.720      1.132   18.31  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##              edf Ref.df    F  p-value
## s(sg_putt)  1.000  1.000 57.70 < 2e-16 ***
## s(sg_ott)   2.232  2.831 12.45 3.73e-06 ***
## s(sg_app)   1.000  1.000 46.16 < 2e-16 ***
## s(sg_arg)   1.364  1.653 15.20 0.000147 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.711   Deviance explained = 73.3%
## GCV = 105.3   Scale est. = 96.04       n = 75
```

```
plot(reg_gam)
```





```
library(broom)
```

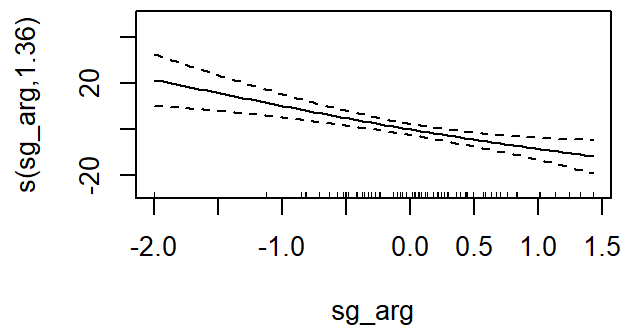
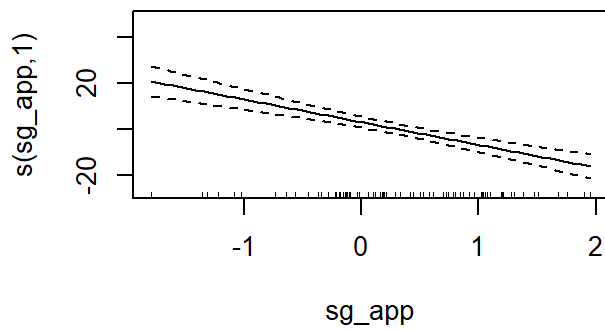
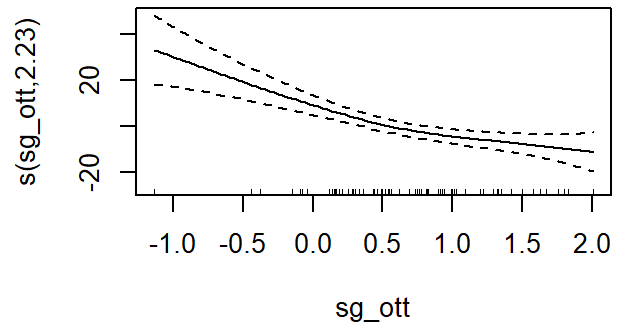
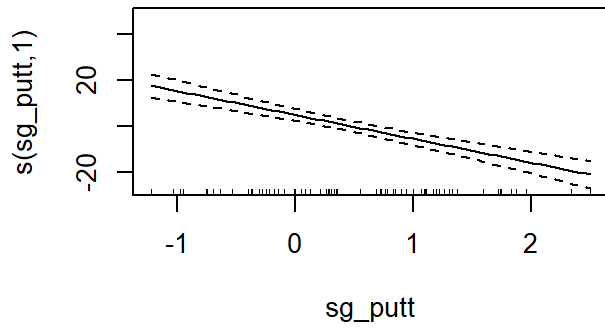
```
reg1_table <- tidy(reg1) %>%  
  dplyr::select(term, estimate, std.error, statistic, p.value)
```

```
reg1_table
```

```
## # A tibble: 5 × 5  
##   term          estimate std.error statistic  p.value  
##   <chr>          <dbl>     <dbl>     <dbl>   <dbl>  
## 1 (Intercept)    37.6       1.99      18.9 3.47e-29  
## 2 sg_putt      -10.3       1.41      -7.26 4.20e-10  
## 3 sg_ott       -11.8       2.15      -5.49 6.10e- 7  
## 4 sg_app       -10.2       1.49      -6.82 2.71e- 9  
## 5 sg_arg        -8.68       2.08      -4.18 8.36e- 5
```

```
par(mfrow = c(2, 2))
```

```
plot(  
  reg_gam,  
  shade = TRUE,  
  shade.col = "lightgray",  
  residuals = FALSE,  
  seWithMean = TRUE,  
  main = ""  
)
```



```
par(mfrow = c(1, 1))
```

## #15. K-MEANS CLUSTERING

```
library(tidyverse)
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```



```
# 6. Attach cluster labels back to bryson_tourn
bryson_tourn$cluster_kmeans <- NA_integer_      # pre-fill with NA
bryson_tourn$cluster_kmeans[complete_idx] <- kmeans_model$cluster

head(bryson_tourn)
```

```
## # A tibble: 6 × 9
##   `tournament id` date      pos sg_putt sg_ott sg_app sg_arg sg_total
##   <dbl> <date>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 2250 2015-06-14 45 0.31 0.29 0.72 -0.51 0.81
## 2 2726 2016-03-16 2  NaN  NaN  NaN  NaN  NaN
## 3 2493 2016-04-10 21 NaN  NaN  NaN  NaN  NaN
## 4 2494 2016-04-17 4 -0.01 0.25 1.38 1.02 2.65
## 5 2499 2016-06-05 38 -0.74 0.12 -0.02 1.43 0.79
## 6 2501 2016-06-19 15 NaN  NaN  NaN  NaN  NaN
## # i 1 more variable: cluster_kmeans <int>
```

## #16. HIERARCHICAL CLUSTERING

```
# 16. HIERARCHICAL CLUSTERING (SG PATTERNS)

# If this chunk runs on its own, recreate complete_idx & cluster_mat:
complete_idx <- complete.cases(
  bryson_tourn[, c("sg_putt", "sg_ott", "sg_app", "sg_arg", "sg_total")]
)

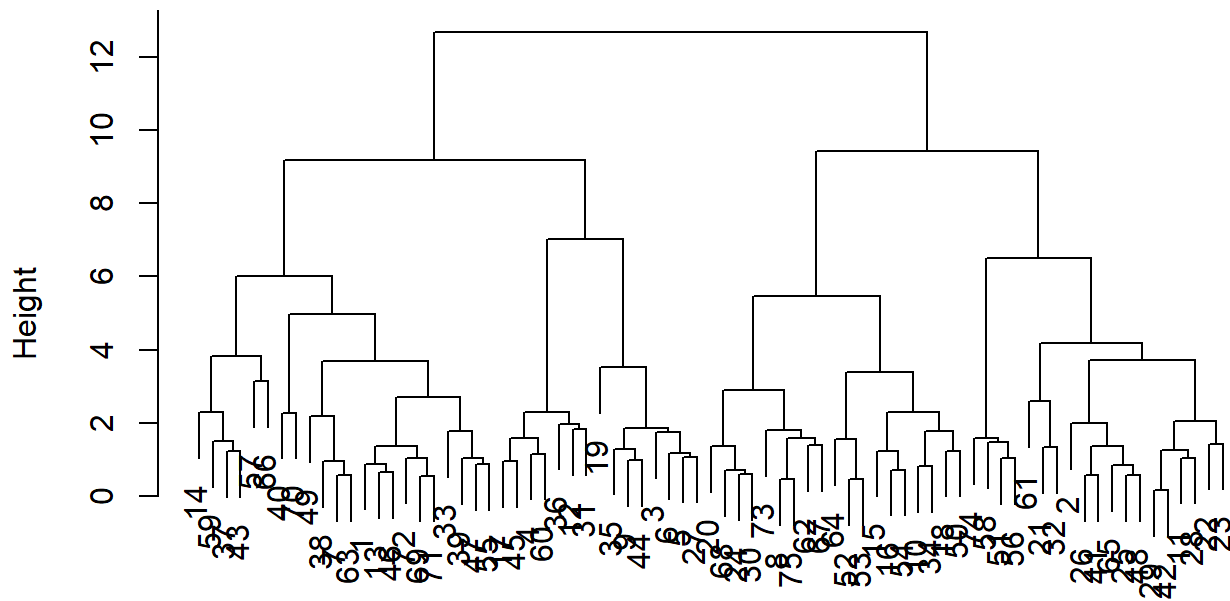
cluster_mat <- bryson_tourn[complete_idx, ] %>%
  dplyr::select(sg_putt, sg_ott, sg_app, sg_arg, sg_total) %>%
  scale()

# Distance + hierarchical clustering
dist_matrix <- dist(cluster_mat, method = "euclidean")
hc <- hclust(dist_matrix, method = "ward.D2")

# Plot dendrogram
plot(hc, main = "Hierarchical Clustering of SG Profiles", xlab = "", sub = "")
```



## Hierarchical Clustering of SG Profiles



```
# Cut into 3 clusters (for example)
hc_clusters <- cutree(hc, k = 3)

# Add to bryson_tourn (only for complete rows)
bryson_tourn$cluster_hc <- NA_integer_
bryson_tourn$cluster_hc[complete_idx] <- hc_clusters

# Compare to k-means on the same complete cases
table(
  KMeans = bryson_tourn$cluster_kmeans[complete_idx],
  HC      = bryson_tourn$cluster_hc[complete_idx]
)
```

```
##      HC
## KMeans  1  2  3
##      1 24  0  7
##      2 10  4  3
##      3  3 15  9
```

## 17. DTW CLUSTERING OF SG COMPONENTS

```
library(TSclust)
```

```
## Loading required package: pdc
```

```
library(dtw)
```

```
## Loading required package: proxy
```

```
##  
## Attaching package: 'proxy'
```

```
## The following object is masked from 'package:Matrix':  
##  
##    as.matrix
```

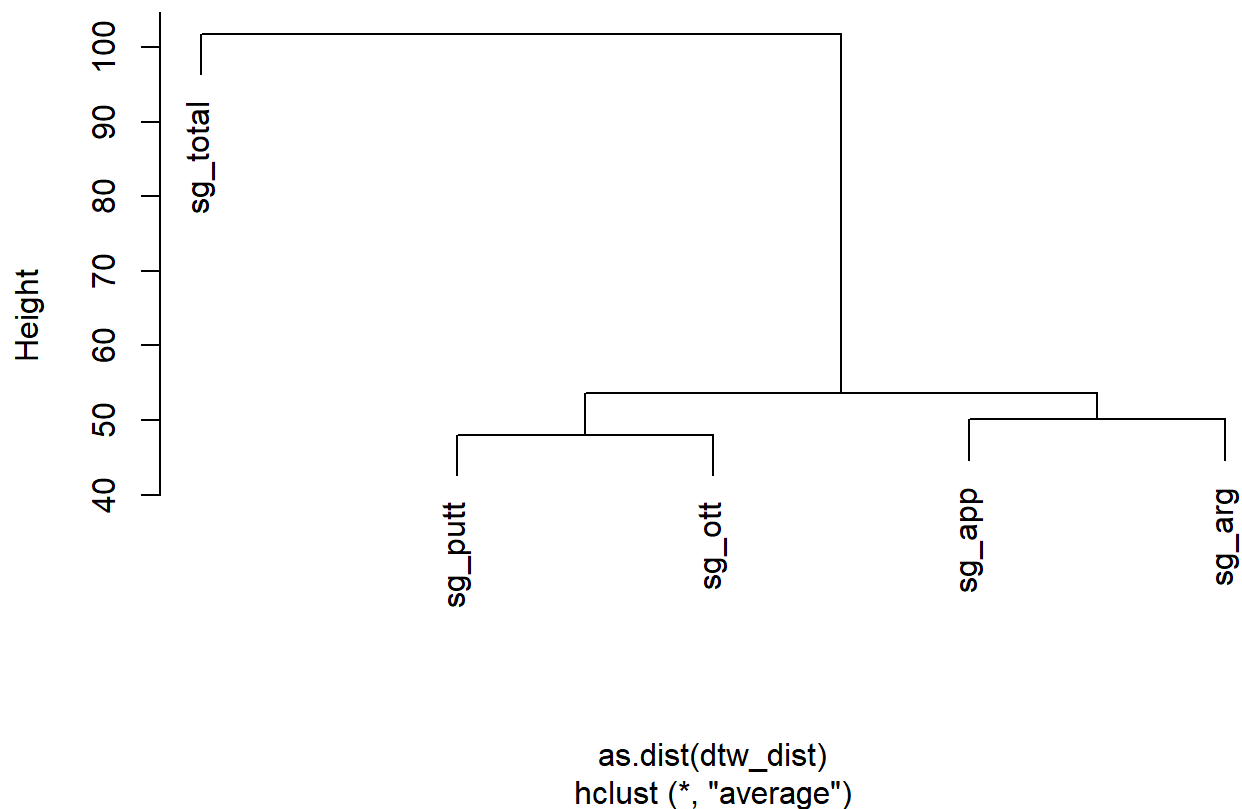
```
## The following objects are masked from 'package:stats':  
##  
##    as.dist, dist
```

```
## The following object is masked from 'package:base':  
##  
##    as.matrix
```

```
## Loaded dtw v1.23-1. See ?dtw for help, citation("dtw") for use in publication.
```

```
# 1. Identify complete rows across ALL SG variables  
idx_comp <- complete.cases(  
  bryson_tourn[, c("sg_putt", "sg_ott", "sg_app", "sg_arg", "sg_total")]  
)  
  
# 2. Build matrix using only complete rows (DTW cannot handle NA)  
sg_mat <- bryson_tourn[idx_comp, ] %>%  
  dplyr::select(sg_putt, sg_ott, sg_app, sg_arg, sg_total) %>%  
  t() # rows = SG components, cols = time  
  
# 3. Compute DTW distance  
dtw_dist <- TSclust::diss(sg_mat, "DTW")  
  
# 4. Hierarchical clustering  
hc_dtw <- hclust(as.dist(dtw_dist), method = "average")  
  
plot(hc_dtw, main = "DTW Hierarchical Clustering of SG Components")
```

## DTW Hierarchical Clustering of SG Components



```
# 5. Choose 2 clusters (or test others)
sg_component_clusters <- cutree(hc_dtw, k = 2)
sg_component_clusters
```

```
## sg_putt sg_ott sg_app sg_arg sg_total
##      1      1      1      1      2
```

## 18. SUBSEQUENCE CLUSTERING (SG\_TOTAL HOT/COLD STREAKS)

```

library(dtw)
library(ggplot2)

series <- bryson_tourn$sg_total
series <- series[!is.na(series)] # drop NAs for subsequences

window <- 5
subseq <- embed(series, window)[, window:1]

subseq_dist <- dist(subseq, method = "DTW")

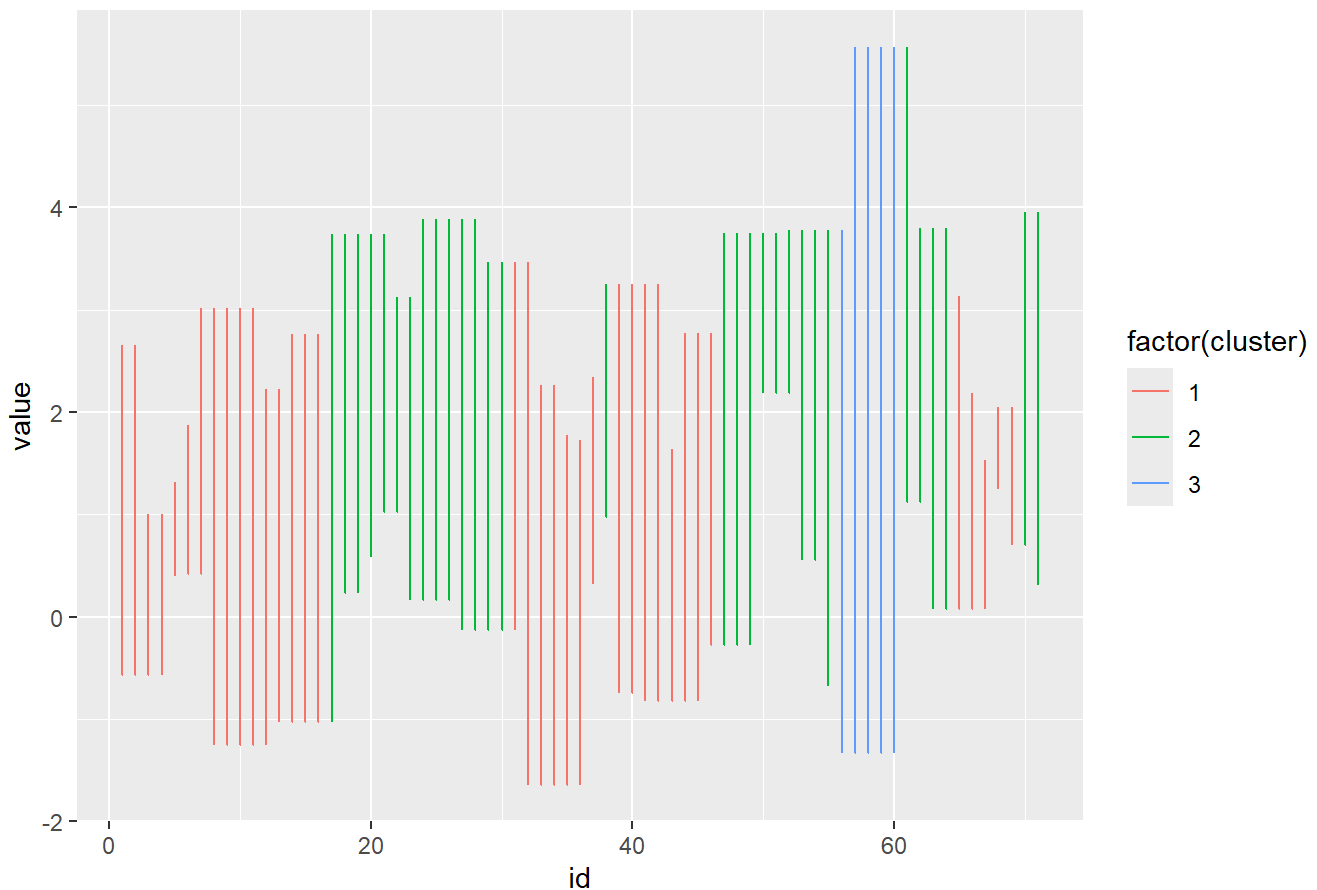
set.seed(123)
subseq_kmeans <- kmeans(subseq_dist, centers = 3)

df_sub <- data.frame(
  id      = rep(1:nrow(subseq), each = window),
  value   = as.vector(t(subseq)),
  cluster = rep(subseq_kmeans$cluster, each = window)
)

ggplot(df_sub, aes(x = id, y = value, group = id, color = factor(cluster))) +
  geom_line() +
  ggtitle("Subsequence Clustering: SG_total Hot/Cold Streaks")

```

Subsequence Clustering: SG\_total Hot/Cold Streaks



# 19. TIME-POINT CLUSTERING

```
library(factoextra)

# Only use rows where sg_total is not NA
idx_total <- !is.na(bryson_tourn$sg_total)

sg_scaled <- scale(bryson_tourn$sg_total[idx_total, drop = FALSE])

# Choose k via elbow/silhouette if you want:
# fviz_nbclust(as.data.frame(sg_scaled), kmeans, method = "wss")
# fviz_nbclust(as.data.frame(sg_scaled), kmeans, method = "silhouette")

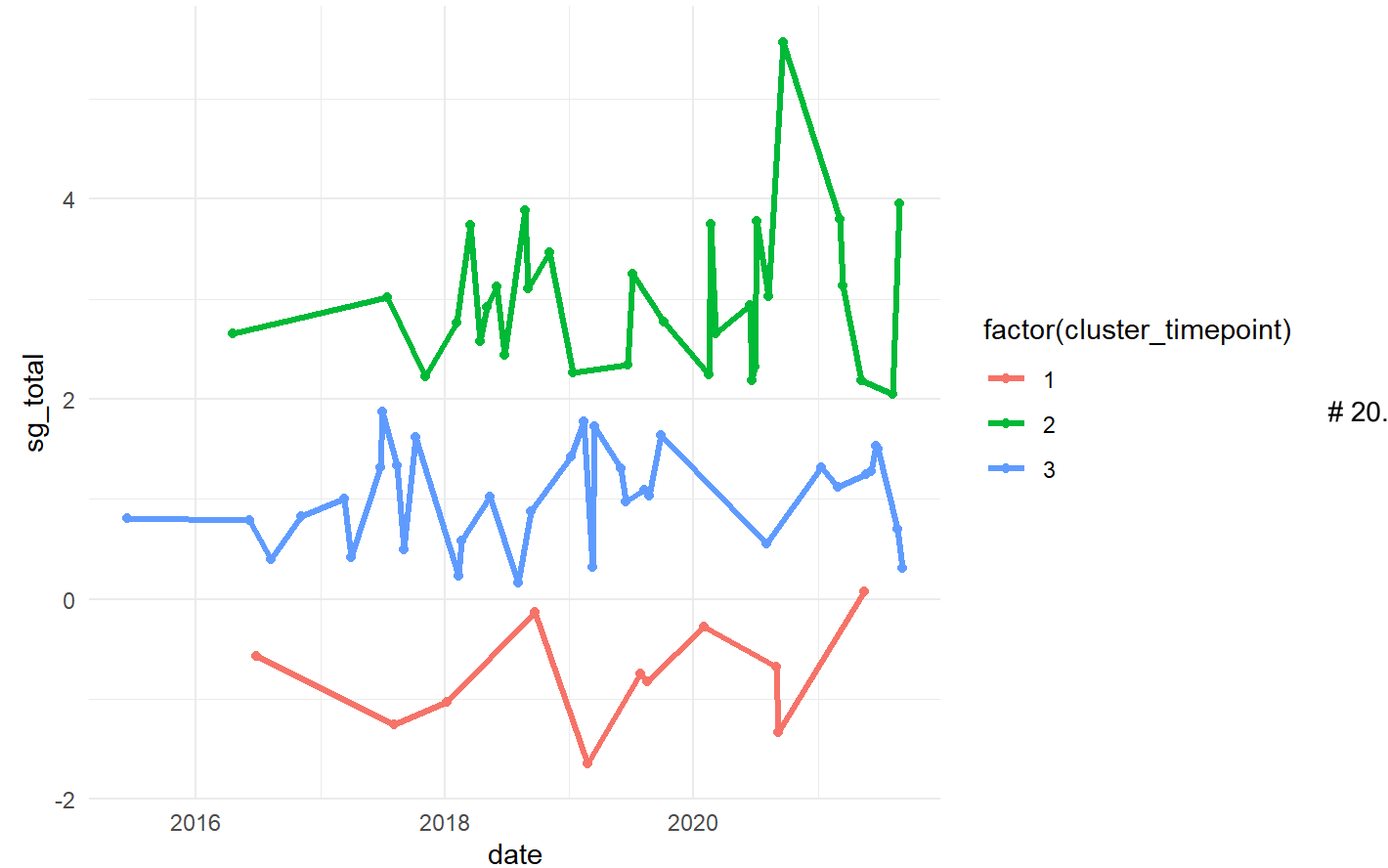
set.seed(123)
tp_kmeans <- kmeans(sg_scaled, centers = 3, nstart = 25)

# Attach clusters back to full data
bryson_tourn$cluster_timepoint <- NA_integer_
bryson_tourn$cluster_timepoint[idx_total] <- tp_kmeans$cluster

# Plot (only points with sg_total)
ggplot(bryson_tourn[idx_total, ],
       aes(x = date, y = sg_total, color = factor(cluster_timepoint))) +
  geom_line(size = 1.1) +
  geom_point() +
  ggtitle("Time-Point Clustering: SG_total Regimes") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Time-Point Clustering: SG\_total Regimes



CROSS-VALIDATION FOR ARIMA FORECASTs

```

library(forecast)
library(tidyverse)

cv_arma_all <- function(series, name, seasonal = FALSE) {
  library(forecast)

  # Turn into ts (tournament index as time)
  y_ts <- ts(as.numeric(series), frequency = 1)
  n <- length(y_ts)

  # Forecast horizon
  h <- 5

  # ----- 1. Simple 80-20 train/test split -----
  split <- floor(0.8 * n)
  train_tt <- window(y_ts, end = split)
  test_tt <- window(y_ts, start = split + 1)

  fit_tt <- auto.arima(train_tt, seasonal = seasonal)
  fc_tt <- forecast(fit_tt, h = length(test_tt))

  acc_tt <- forecast::accuracy(fc_tt, test_tt)
  tt_rmse <- acc_tt["Test set", "RMSE"]
  tt_mape <- acc_tt["Test set", "MAPE"]

  # ----- 2. Rolling-origin CV (h-step ahead) -----
  roll_err <- tsCV(
    y_ts,
    forecastfunction = function(x, h) {
      forecast(auto.arima(x, seasonal = seasonal), h = h)
    },
    h = h
  )
  roll_rmse <- sqrt(mean(roll_err^2, na.rm = TRUE))
  roll_mape <- mean(abs(roll_err / y_ts), na.rm = TRUE) * 100

  # ----- 3. Expanding-window CV (1-step ahead) -----
  exp_err <- tsCV(
    y_ts,
    forecastfunction = function(x, h) {
      forecast(auto.arima(x, seasonal = seasonal), h = h)
    },
    h = 1
  )
  exp_rmse <- sqrt(mean(exp_err^2, na.rm = TRUE))
  exp_mape <- mean(abs(exp_err / y_ts), na.rm = TRUE) * 100

  # ----- 4. Blocked k-fold CV -----
  k <- 5
  block_size <- floor(n / k)
  blk_rmse <- numeric(k)
  blk_mape <- numeric(k)

```

```

for (i in seq_len(k)) {
  start_i <- (i - 1) * block_size + 1
  end_i    <- if (i == k) n else i * block_size

  train_idx <- setdiff(seq_len(n), start_i:end_i)
  train_blk <- y_ts[train_idx]
  test_blk  <- y_ts[start_i:end_i]

  fit_blk <- auto.arima(train_blk, seasonal = seasonal)
  fc_blk  <- forecast(fit_blk, h = length(test_blk))

  acc_blk      <- forecast::accuracy(fc_blk, test_blk)
  blk_rmse[i]  <- acc_blk["Test set", "RMSE"]
  blk_mape[i]  <- acc_blk["Test set", "MAPE"]
}

tibble::tibble(
  series_name = name,
  method      = c("Train-test", "Rolling", "Expanding", "Blocked"),
  RMSE        = c(tt_rmse, roll_rmse, exp_rmse, mean(blk_rmse)),
  MAPE        = c(tt_mape, roll_mape, exp_mape, mean(blk_mape))
)
}

cv_all <- bind_rows(
  cv_arma_all(bryson_tourn$sg_putt, "SG Putting"),
  cv_arma_all(bryson_tourn$sg_ott,  "SG Off-the-Tee"),
  cv_arma_all(bryson_tourn$sg_app,  "SG Approach"),
  cv_arma_all(bryson_tourn$sg_arg,  "SG Around-the-Green"),
  cv_arma_all(bryson_tourn$sg_total, "SG Total"),
  cv_arma_all(bryson_tourn$sg_pos,   "Finishing Position")
)

cv_all

```

```

## # A tibble: 24 × 4
##   series_name    method      RMSE  MAPE
##   <chr>         <chr>    <dbl> <dbl>
## 1 SG Putting    Train-test 0.937 150.
## 2 SG Putting    Rolling    0.897 Inf
## 3 SG Putting    Expanding  0.915 Inf
## 4 SG Putting    Blocked    0.927 Inf
## 5 SG Off-the-Tee Train-test 0.486 30.2
## 6 SG Off-the-Tee Rolling    0.611 112.
## 7 SG Off-the-Tee Expanding  0.589 104.
## 8 SG Off-the-Tee Blocked    0.732 248.
## 9 SG Approach   Train-test 0.706 215.
## 10 SG Approach   Rolling    0.856 274.
## # i 14 more rows

```



# 21. SPECTRAL ANALYSIS OF STROKES-GAINED SERIES

```
library(tidyverse)

# -----
# 1. CLEAN DATA: drop tournaments with ANY missing SG value
# -----
sg_df <- bryson_tourn %>%
  dplyr::select(sg_putt, sg_ott, sg_app, sg_arg, sg_total) %>% # NOTE: dplyr::select
  drop_na()

# Recreate ts objects AFTER dropping tournaments with NA
sg_putt_ts <- ts(sg_df$sg_putt)
sg_ott_ts <- ts(sg_df$sg_ott)
sg_app_ts <- ts(sg_df$sg_app)
sg_arg_ts <- ts(sg_df$sg_arg)
sg_total_ts <- ts(sg_df$sg_total)

# Confirm no missing values remain
colSums(is.na(sg_df))
```

```
## sg_putt sg_ott sg_app sg_arg sg_total
##      0      0      0      0      0
```

```

# -----
# 2. Helper function for spectral analysis of one series
# -----
spec_summary <- function(x, name, spans = c(3, 3)) {
  # make sure no NA and numeric
  x <- stats::na.omit(as.numeric(x))

  if (length(x) < 10) {
    warning(paste("Series", name, "too short for spectral analysis"))
    return(NULL)
  }

  spec_obj <- spectrum(
    x,
    log = "no",
    spans = spans,
    main = paste("Smoothed Spectrum -", name)
  )

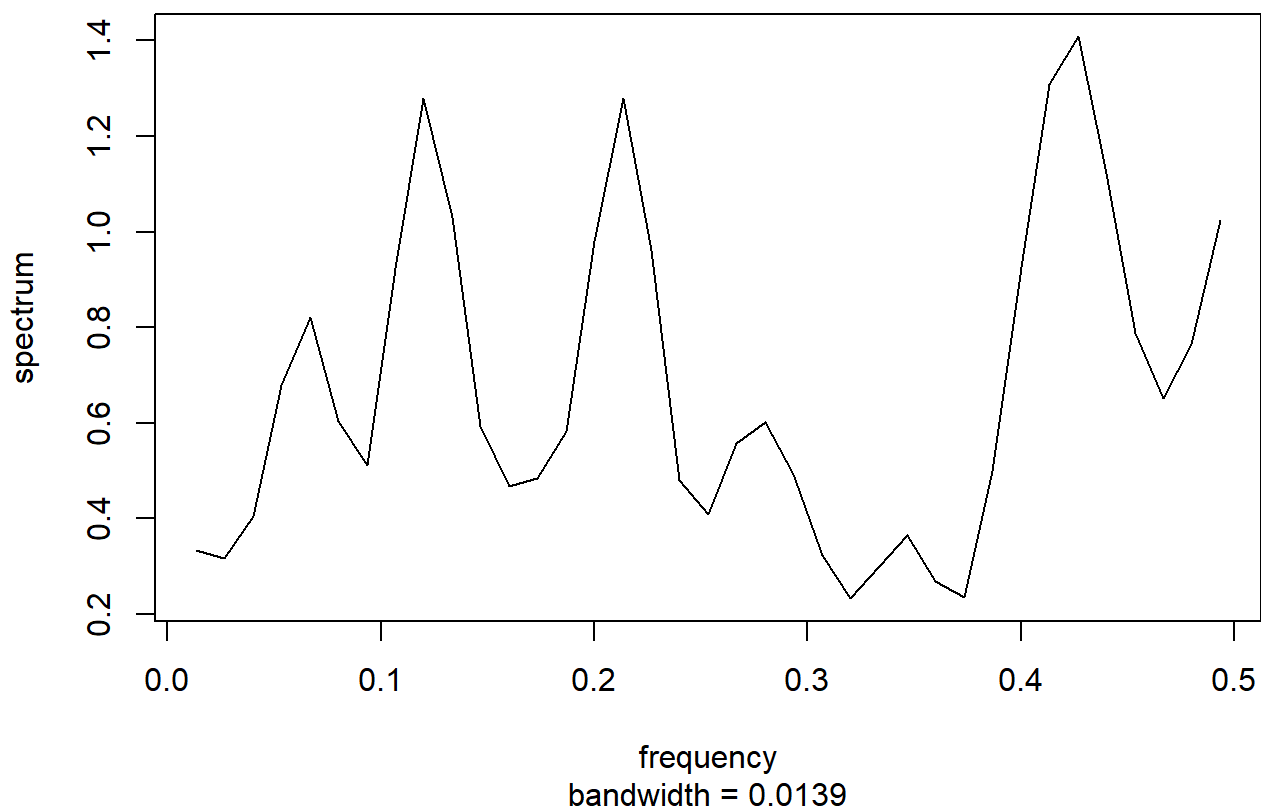
  # Top K peaks in spectral density
  K <- min(5, length(spec_obj$spec))
  ord <- order(spec_obj$spec, decreasing = TRUE)
  top_idx <- ord[1:K]

  data.frame(
    series      = name,
    index       = top_idx,
    frequency    = spec_obj$freq[top_idx],      # cycles per tournament
    period_tourns = 1 / spec_obj$freq[top_idx],  # tournaments per cycle
    density      = spec_obj$spec[top_idx]
  )
}

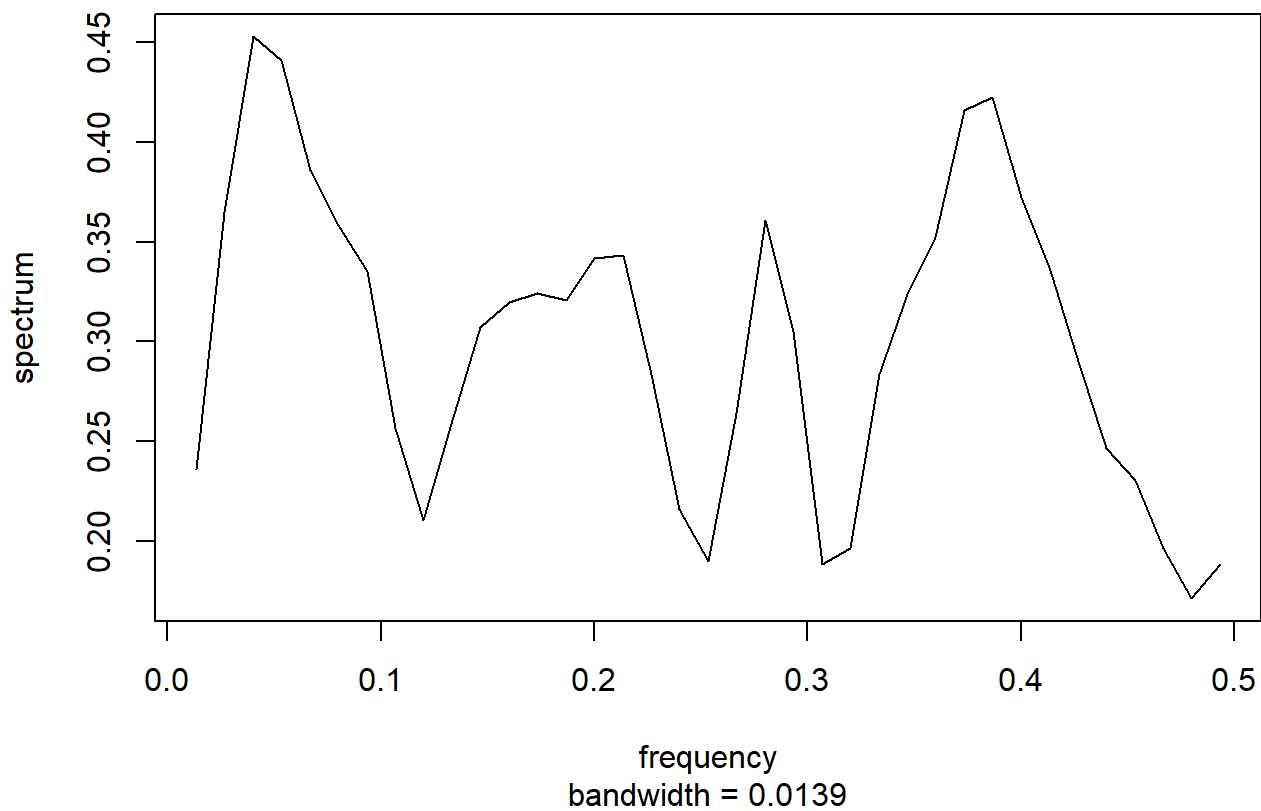
# -----
# 3. Run spectral analysis for each SG component
# -----
spec_list <- list(
  spec_summary(sg_putt_ts, "SG Putting"),
  spec_summary(sg_ott_ts, "SG Off-the-Tee"),
  spec_summary(sg_app_ts, "SG Approach"),
  spec_summary(sg_arg_ts, "SG Around Green"),
  spec_summary(sg_total_ts, "SG Total")
)

```

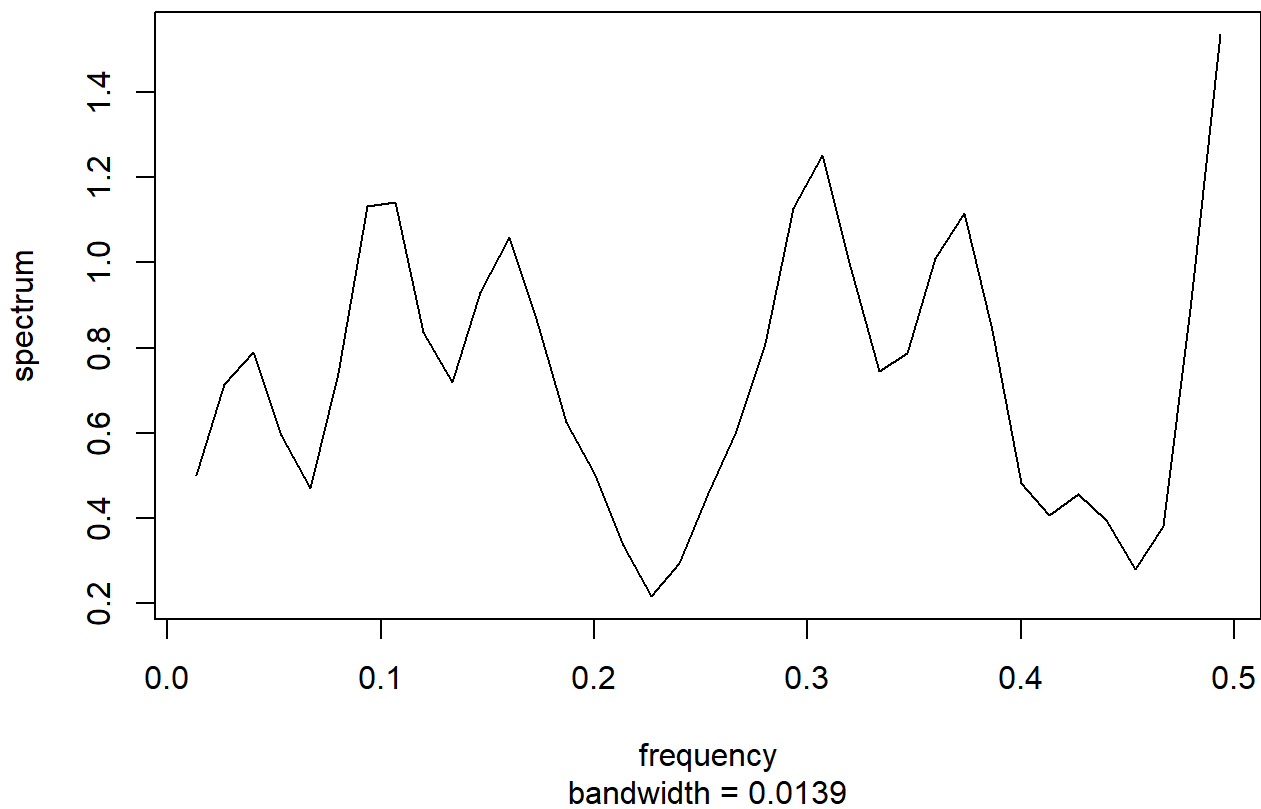
### Smoothed Spectrum - SG Putting



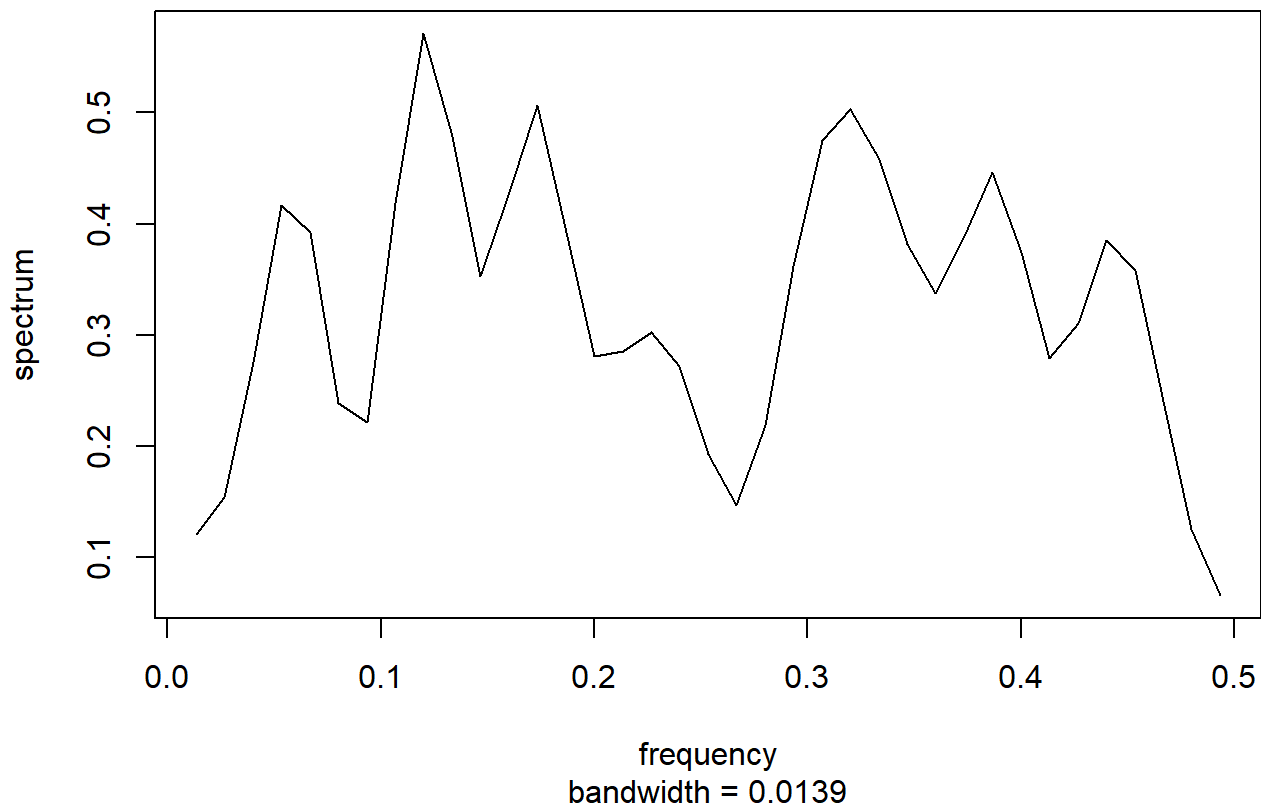
### Smoothed Spectrum - SG Off-the-Tee



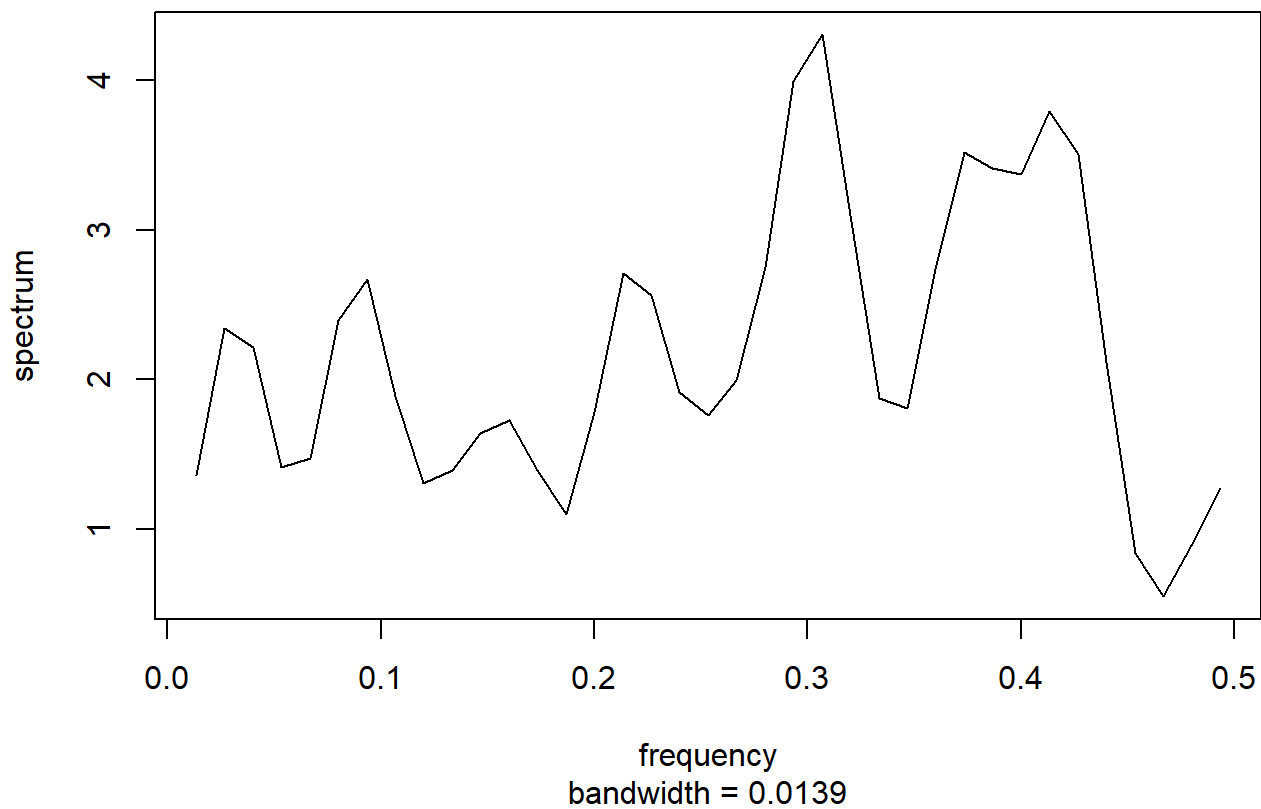
### Smoothed Spectrum - SG Approach



### Smoothed Spectrum - SG Around Green



## Smoothed Spectrum - SG Total



```
# Remove any NULLs just in case and bind rows
spec_results <- do.call(rbind, spec_list[!sapply(spec_list, is.null)])

# View table of dominant cycles (across all series)
spec_results
```

| ##    |                 | series | index      | frequency | period_tourns | density |
|-------|-----------------|--------|------------|-----------|---------------|---------|
| ## 1  | SG Putting      | 32     | 0.42666667 | 2.343750  | 1.4084727     |         |
| ## 2  | SG Putting      | 31     | 0.41333333 | 2.419355  | 1.3092333     |         |
| ## 3  | SG Putting      | 16     | 0.21333333 | 4.687500  | 1.2803957     |         |
| ## 4  | SG Putting      | 9      | 0.12000000 | 8.333333  | 1.2786206     |         |
| ## 5  | SG Putting      | 33     | 0.44000000 | 2.272727  | 1.1220222     |         |
| ## 6  | SG Off-the-Tee  | 3      | 0.04000000 | 25.000000 | 0.4530301     |         |
| ## 7  | SG Off-the-Tee  | 4      | 0.05333333 | 18.750000 | 0.4414092     |         |
| ## 8  | SG Off-the-Tee  | 29     | 0.38666667 | 2.586207  | 0.4226224     |         |
| ## 9  | SG Off-the-Tee  | 28     | 0.37333333 | 2.678571  | 0.4160517     |         |
| ## 10 | SG Off-the-Tee  | 5      | 0.06666667 | 15.000000 | 0.3858792     |         |
| ## 11 | SG Approach     | 37     | 0.49333333 | 2.027027  | 1.5353842     |         |
| ## 12 | SG Approach     | 23     | 0.30666667 | 3.260870  | 1.2526218     |         |
| ## 13 | SG Approach     | 8      | 0.10666667 | 9.375000  | 1.1423870     |         |
| ## 14 | SG Approach     | 7      | 0.09333333 | 10.714286 | 1.1331177     |         |
| ## 15 | SG Approach     | 22     | 0.29333333 | 3.409091  | 1.1287004     |         |
| ## 16 | SG Around Green | 9      | 0.12000000 | 8.333333  | 0.5709150     |         |
| ## 17 | SG Around Green | 13     | 0.17333333 | 5.769231  | 0.5063120     |         |
| ## 18 | SG Around Green | 24     | 0.32000000 | 3.125000  | 0.5027034     |         |
| ## 19 | SG Around Green | 10     | 0.13333333 | 7.500000  | 0.4788168     |         |
| ## 20 | SG Around Green | 23     | 0.30666667 | 3.260870  | 0.4755257     |         |
| ## 21 | SG Total        | 23     | 0.30666667 | 3.260870  | 4.3066704     |         |
| ## 22 | SG Total        | 22     | 0.29333333 | 3.409091  | 3.9960542     |         |
| ## 23 | SG Total        | 31     | 0.41333333 | 2.419355  | 3.7955504     |         |
| ## 24 | SG Total        | 28     | 0.37333333 | 2.678571  | 3.5210457     |         |
| ## 25 | SG Total        | 32     | 0.42666667 | 2.343750  | 3.5096747     |         |

## 22. NEURAL NETWORK MODELS FOR BRYSON (SG TOTAL AS EXAMPLE)

```
library(forecast)
library(Metrics)
```

```
##
## Attaching package: 'Metrics'
```

```
## The following object is masked from 'package:tsDyn':
##
## mse
```

```
## The following object is masked from 'package:forecast':
##
## accuracy
```

```

library(dplyr)

# Clean tournament-level data
library(dplyr)
library(tidyr) # for drop_na()

bryson_nn <- bryson_tourn %>%
  dplyr::select(sg_putt, sg_ott, sg_app, sg_arg, sg_total, pos) %>%
  tidyr::drop_na()

nnar_one <- function(y, series_name, h = 5, p = 4, size = 5) {
  y <- as.numeric(y)
  y_ts <- ts(y, frequency = 1)

  n <- length(y_ts)
  train_size <- n - h

  train <- window(y_ts, end = train_size)
  test <- window(y_ts, start = train_size + 1)

  fit <- nnetar(train, p = p, size = size)
  fc <- forecast(fit, h = length(test))

  data.frame(
    series = series_name,
    RMSE = rmse(test, fc$mean),
    MAPE = mape(test, fc$mean),
    h = length(test)
  )
}

nnar_results <- bind_rows(
  nnar_one(bryson_nn$sg_putt, "SG Putting"),
  nnar_one(bryson_nn$sg_ott, "SG Off-the-Tee"),
  nnar_one(bryson_nn$sg_app, "SG Approach"),
  nnar_one(bryson_nn$sg_arg, "SG Around-the-Green"),
  nnar_one(bryson_nn$sg_total, "SG Total"),
  nnar_one(bryson_nn$pos, "Finishing Position")
)

nnar_results

```

| ##   | series              | RMSE       | MAPE      | h |
|------|---------------------|------------|-----------|---|
| ## 1 | SG Putting          | 1.0543333  | 1.3821017 | 5 |
| ## 2 | SG Off-the-Tee      | 1.0298409  | 0.4018960 | 5 |
| ## 3 | SG Approach         | 0.9389566  | 4.0118211 | 5 |
| ## 4 | SG Around-the-Green | 0.7198648  | 2.1127674 | 5 |
| ## 5 | SG Total            | 1.3286328  | 0.6405047 | 5 |
| ## 6 | Finishing Position  | 17.8564599 | 3.6680372 | 5 |

# 23. HIDDEN MARKOV MODEL: STREAKY PUTTING REGIMES

```
library(depmixS4)
```

```
## Loading required package: nnet
```

```
##
## Attaching package: 'nnet'
```

```
## The following object is masked from 'package:mgcv':
##
##      multinom
```

```
## Loading required package: Rsolnp
```

```
library(dplyr)

# -----
# 1. Tournament-Level SG putting series
# -----
dat <- bryson_tourn %>%
  arrange(date) %>%           # ensure tournaments are in time order
  filter(!is.na(sg_putt)) %>% # drop missing SG putting
  dplyr::select(date, sg_putt) # explicit namespace to avoid masking

nrow(dat)
```

```
## [1] 75
```

```
summary(dat$sg_putt)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -1.2200 -0.1600   0.3600   0.4859  1.1050   2.5000
```



```
# -----
# 2. Fit 2-state and 3-state Gaussian HMMs
# -----
set.seed(123)

hmm2 <- depmix(
  sg_putt ~ 1,
  family = gaussian(),
  nstates = 2,
  data = dat
)
hmm2_fit <- fit(hmm2, verbose = FALSE)
```

```
## converged at iteration 146 with logLik: -92.21386
```

```
hmm3 <- depmix(
  sg_putt ~ 1,
  family = gaussian(),
  nstates = 3,
  data = dat
)
hmm3_fit <- fit(hmm3, verbose = FALSE)
```

```
## converged at iteration 207 with logLik: -87.04609
```

```
# Compare model fit
markov_aic <- data.frame(
  Model = c("HMM 2-state", "HMM 3-state"),
  AIC = c(AIC(hmm2_fit), AIC(hmm3_fit))
)
markov_aic
```

```
##           Model      AIC
## 1 HMM 2-state 198.4277
## 2 HMM 3-state 202.0922
```

```
# -----
# 3. Inspect preferred model (3-state)
# -----
summary(hmm3_fit)
```

```
## Initial state probabilities model
## pr1 pr2 pr3
## 0 0 1
##
## Transition matrix
##      toS1 toS2 toS3
## fromS1 0.714 0.286 0.000
## fromS2 1.000 0.000 0.000
## fromS3 0.000 0.136 0.864
##
## Response parameters
## Resp 1 : gaussian
##      Re1.(Intercept) Re1.sd
## St1      0.872 0.679
## St2     -0.507 0.398
## St3     -0.112 0.427
```

```
# -----
# 4. Transition matrix (robust extraction)
# -----
nstates <- 3
pars3 <- getpars(hmm3_fit)

tm3 <- matrix(
  pars3[(nstates + 1):(nstates + nstates^2)],
  nrow = nstates,
  byrow = TRUE
)

rownames(tm3) <- paste0("From State ", 1:nstates)
colnames(tm3) <- paste0("To State ", 1:nstates)
tm3
```

```
##              To State 1   To State 2   To State 3
## From State 1 7.140625e-01 2.859375e-01 6.237942e-33
## From State 2 9.999996e-01 3.790189e-07 5.401677e-51
## From State 3 1.187837e-27 1.356745e-01 8.643255e-01
```

```
# -----
# 5. State means, SDs, and most likely state sequence
# -----
post3 <- posterior(hmm3_fit)
```

```
## Warning in .local(object, ...): Argument 'type' not specified and will default
## to 'viterbi'. This default may change in future releases of depmixS4. Please
## see ?posterior for alternative options.
```

```

dat$state3 <- post3$state

state_means3 <- tapply(dat$sg_putt, dat$state3, mean)
state_sds3 <- tapply(dat$sg_putt, dat$state3, sd)

state_summary3 <- data.frame(
  State = as.integer(names(state_means3)),
  Mean_SG_Putt = as.numeric(state_means3),
  SD_SG_Putt = as.numeric(state_sds3)
)
state_summary3

```

```

##   State Mean_SG_Putt SD_SG_Putt
## 1     1     0.925600  0.6361785
## 2     2    -0.564375  0.3526465
## 3     3    -0.090000  0.4484139

```

```

# -----
# 6. Plot regimes over time
# -----

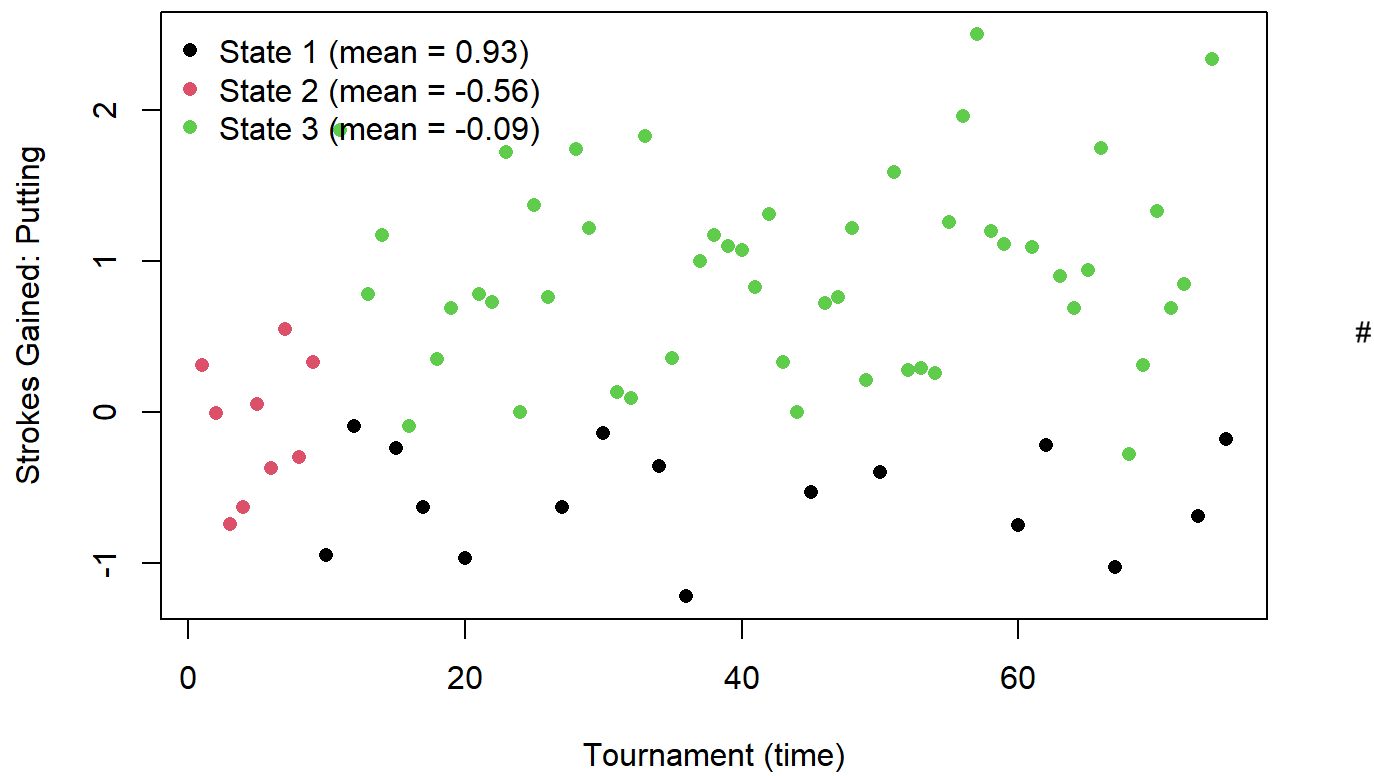
ord <- order(state_summary3$Mean_SG_Putt)
state_map <- setNames(1:nstates, state_summary3$State[ord])
dat$state3_ordered <- state_map[as.character(dat$state3)]

plot(
  dat$sg_putt,
  col = dat$state3_ordered,
  pch = 16,
  xlab = "Tournament (time)",
  ylab = "Strokes Gained: Putting",
  main = "Hidden Markov Regimes for Bryson's SG Putting"
)

legend(
  "topleft",
  legend = paste0(
    "State ", state_summary3$State,
    " (mean = ", sprintf("%.2f", state_summary3$Mean_SG_Putt), ")"
  ),
  col = 1:nstates,
  pch = 16,
  bty = "n"
)

```

Hidden Markov Regimes for Bryson's SG Putting



PLAYER CLUSTERING: WHERE DOES BRYSON FALL?

```

library(tidyverse)
library(cluster)
library(factoextra)

# ---- FIX: ensure pga_raw exists in the knit session ----
if (!exists("pga_raw")) {
  if (exists("bryson_raw")) {
    pga_raw <- bryson_raw
  } else if (file.exists("ASA All PGA Raw Data - Tourn Level.csv")) {
    pga_raw <- readr::read_csv("ASA All PGA Raw Data - Tourn Level.csv", show_col_types = FALSE)
  } else {
    stop("Missing data: create `pga_raw` (or `bryson_raw`) earlier, or put the CSV in the working directory.")
  }
}

# ---- 1) Tournament-Level aggregation ----
pga_tourn <- pga_raw %>%
  dplyr::mutate(date = as.Date(date)) %>%
  dplyr::filter(!is.na(player), !is.na(`tournament id`)) %>%
  dplyr::group_by(player, `tournament id`) %>%
  dplyr::summarise(
    date      = suppressWarnings(min(date, na.rm = TRUE)),
    pos       = suppressWarnings(first(na.omit(pos))),
    sg_putt   = mean(sg_putt, na.rm = TRUE),
    sg_ott    = mean(sg_ott, na.rm = TRUE),
    sg_app    = mean(sg_app, na.rm = TRUE),
    sg_arg    = mean(sg_arg, na.rm = TRUE),
    sg_total  = mean(sg_total, na.rm = TRUE),
    .groups   = "drop"
  )

# ---- 2) Player-Level profiles (one row per player) ----
min_tourn <- 20

player_profiles <- pga_tourn %>%
  dplyr::group_by(player) %>%
  dplyr::summarise(
    n_tourn   = sum(complete.cases(sg_putt, sg_ott, sg_app, sg_arg, sg_total)),
    sg_putt   = mean(sg_putt, na.rm = TRUE),
    sg_ott    = mean(sg_ott, na.rm = TRUE),
    sg_app    = mean(sg_app, na.rm = TRUE),
    sg_arg    = mean(sg_arg, na.rm = TRUE),
    sg_total  = mean(sg_total, na.rm = TRUE),
    .groups   = "drop"
  ) %>%
  dplyr::filter(n_tourn >= min_tourn) %>%
  tidyr::drop_na()

# ---- 3) Scale features ----
features <- player_profiles %>%
  dplyr::select(sg_putt, sg_ott, sg_app, sg_arg, sg_total)

```

```

X <- scale(features)

# ---- 4) K-means clustering ----
set.seed(123)
k <- 4
km <- kmeans(X, centers = k, nstart = 50)

player_profiles <- player_profiles %>%
  dplyr::mutate(cluster_kmeans = km$cluster)

# ---- 5) Where does Bryson fall? ----
bryson_name <- "Bryson DeChambeau"

bryson_row <- player_profiles %>%
  dplyr::filter(player == bryson_name)

if (nrow(bryson_row) == 0) {

  message(
    "Bryson not found among eligible players. ",
    "Try lowering min_tourn (currently = ", min_tourn, ")."
  )

} else {

  bryson_x <- X[player_profiles$player == bryson_name, , drop = FALSE]
  bryson_cluster <- bryson_row$cluster_kmeans[1]
  centroid <- km$centers[bryson_cluster, , drop = FALSE]

  dist_to_centroid <- sqrt(sum((bryson_x - centroid)^2))

  cat("\n=== BRYSON CLUSTER RESULT ===\n")

  base::print(
    bryson_row %>%
      dplyr::select(
        player, n_tourn,
        sg_putt, sg_ott, sg_app, sg_arg, sg_total,
        cluster_kmeans
      )
  )

  cat(
    "\nDistance to assigned cluster centroid (scaled space): ",
    round(dist_to_centroid, 4), "\n\n"
  )
}

```

```
##
## === BRYSON CLUSTER RESULT ===
## # A tibble: 1 × 8
##   player          n_tourn sg_putt sg_ott sg_app  sg_arg sg_total cluster_kmeans
##   <chr>          <int>   <dbl> <dbl> <dbl>   <dbl>   <dbl>         <int>
## 1 Bryson DeChambe...    104  0.0482  0.627  0.152 -0.0137   0.814           4
##
## Distance to assigned cluster centroid (scaled space): 1.3943
```

```
# ---- 6) Outputs professors like ----
```

```
# A) Cluster sizes
base::print(table(player_profiles$cluster_kmeans))
```

```
##
##   1    2    3    4
## 33 109  98  92
```

```
# B) Cluster centers (back on original SG scale)
centers_scaled <- km$centers

centers_unscaled <- sweep(centers_scaled, 2, attr(X, "scaled:scale"), `*`)
centers_unscaled <- sweep(centers_unscaled, 2, attr(X, "scaled:center"), `+`)

centers_unscaled <- as.data.frame(centers_unscaled)
centers_unscaled$cluster <- 1:k

base::print(
  centers_unscaled %>%
    dplyr::select(cluster, everything())
)
```

```
##   cluster    sg_putt    sg_ott    sg_app    sg_arg    sg_total
## 1      1 -0.24072619 -0.63579607 -0.6450631 -0.16539056 -1.6805841
## 2      2 -0.28964162  0.02437879 -0.2533068 -0.20672832 -0.7215462
## 3      3  0.01399079 -0.26156595 -0.1965700  0.07425362 -0.3658051
## 4      4 -0.11512374  0.23551092  0.2358297  0.02272542  0.3847141
```

```
#AIC
```

```

# ---- helpers (robust) ----
safe_AIC <- function(obj) {
  out <- tryCatch(AIC(obj), error = function(e) NA_real_)
  as.numeric(out)
}

safe_uGARCH_AIC <- function(fit) {
  # rugarch uses infocriteria(); first element is typically AIC
  out <- tryCatch(infocriteria(fit)[1], error = function(e) NA_real_)
  as.numeric(out)
}

# ---- collect AICs from every model that supports it ----
aic_all <- dplyr::bind_rows(

  # -----
  # A) SG PUTTING UNIVARIATE MODELS (from sections 5-11)
  # -----
  tibble::tibble(
    group = "SG Putting univariate",
    model = c("Naive (Arima 0,0,0 no mean)",
              "Mean (Arima 0,0,0 mean)",
              "Drift (Arima 0,0,0 drift)",
              "ETS",
              "SARIMA (auto.arima on sg_diff1)"),
    AIC = c(
      safe_AIC(naive_model),
      safe_AIC(mean_model),
      safe_AIC(drift_model),
      safe_AIC(ets_model),
      safe_AIC(sarima_model)
    )
  ),

  # GARCH family (rugarch)
  tibble::tibble(
    group = "SG Putting univariate",
    model = c("GARCH(1,1) normal",
              "GARCH(1,1) t",
              "EGARCH(1,1) t"),
    AIC = c(
      safe_uGARCH_AIC(garch11_fit),
      safe_uGARCH_AIC(garch11_t_fit),
      safe_uGARCH_AIC(egarch11_fit)
    )
  ),

  # TAR + FARIMA
  tibble::tibble(
    group = "SG Putting univariate",
    model = c("TAR m=2",
              "TAR m=3",

```



```

      "TAR m=2 delay=1",
      "FARIMA (arfima)"),
  AIC = c(
    safe_AIC(tar_m2),
    safe_AIC(tar_m3),
    safe_AIC(tar_m2_d1),
    safe_AIC(farima_model)
  )
),

# -----
# B) ARIMAX MODELS
# -----
tibble::tibble(
  group = "ARIMAX",
  model = c("ARIMAX on sg_diff1 with xreg=sg_total",
            "ARIMAX on sg_putt levels with xreg=sg_total"),
  AIC = c(
    safe_AIC(arimax_model),
    safe_AIC(arimax_levels)
  )
),

# -----
# C) FINISHING POSITION MODELS (regressions + GAM)
# -----
tibble::tibble(
  group = "Finishing position models",
  model = c("Linear regression: pos ~ sg_putt + sg_ott + sg_app + sg_arg",
            "Polynomial regression (degree 2)",
            "Spline regression (bs df=4)",
            "GAM: pos ~ s(sg_putt)+s(sg_ott)+s(sg_app)+s(sg_arg)"),
  AIC = c(
    safe_AIC(reg1),
    safe_AIC(reg_poly),
    safe_AIC(reg_spline),
    safe_AIC(reg_gam)
  )
),

# -----
# D) VAR MODEL (pos + SG components)
# -----
tibble::tibble(
  group = "Finishing position models",
  model = "VAR (pos, sg_putt, sg_ott, sg_app, sg_arg)",
  AIC = safe_AIC(var_model)
),

# -----
# E) HMM MODELS (depmixS4)
# -----

```

```
tibble::tibble(
  group = "HMM (SG Putting regimes)",
  model = c("HMM 2-state (Gaussian)",
            "HMM 3-state (Gaussian)"),
  AIC = c(
    safe_AIC(hmm2_fit),
    safe_AIC(hmm3_fit)
  )
)

) %>%
  dplyr::arrange(AIC)

# ---- print the table ----
aic_all
```

```
## # A tibble: 21 × 3
##   group                model                AIC
##   <chr>                <chr>                <dbl>
## 1 SG Putting univariate FARIMA (arfima)        -59.9
## 2 SG Putting univariate TAR m=3                -46.7
## 3 SG Putting univariate TAR m=2                -42.9
## 4 SG Putting univariate TAR m=2 delay=1        -37.6
## 5 SG Putting univariate EGARCH(1,1) t           2.73
## 6 SG Putting univariate GARCH(1,1) normal       2.79
## 7 SG Putting univariate GARCH(1,1) t           2.82
## 8 ARIMAX              ARIMAX on sg_diff1 with xreg=sg_total 140.
## 9 SG Putting univariate Drift (Arima 0,0,0 drift) 147.
## 10 SG Putting univariate SARIMA (auto.arima on sg_diff1) 151.
## # i 11 more rows
```

```
# ---- optional: add a note block for "models that do NOT have AIC" ----
no_aic_models <- tibble::tibble(
  group = c("Clustering", "Clustering", "Clustering", "Neural nets", "CV/Accuracy tools", "Spectral analysis"),
  model = c("K-means clustering (tournament profiles)",
            "Hierarchical clustering (ward.D2)",
            "DTW clustering of SG components",
            "NNAR (nnetar)",
            "tsCV / forecast accuracy comparisons",
            "Spectrum / spectral peaks"),
  note = c("No AIC (not likelihood-based)",
            "No AIC (not likelihood-based)",
            "No AIC (distance-based)",
            "No AIC (different criterion)",
            "Not an AIC model",
            "Not an AIC model")
)

no_aic_models
```

```
## # A tibble: 6 × 3
##   group          model          note
##   <chr>          <chr>          <chr>
## 1 Clustering    K-means clustering (tournament profiles) No AIC (not likeli...
## 2 Clustering    Hierarchical clustering (ward.D2)        No AIC (not likeli...
## 3 Clustering    DTW clustering of SG components          No AIC (distance-b...
## 4 Neural nets   NNAR (nnetar)                          No AIC (different ...
## 5 CV/Accuracy tools tsCV / forecast accuracy comparisons Not an AIC model
## 6 Spectral analysis Spectrum / spectral peaks      Not an AIC model
```

## #Core AIC

```
safe_AIC <- function(obj) {
  out <- tryCatch(AIC(obj), error = function(e) NA_real_)
  as.numeric(out)
}
```

```
models_core <- list(
  ARIMAX_sg_diff1_with_sg_total = arimax_model,
  VAR_pos_and_SG                = var_model,
  Linear_regression_pos_on_SG   = reg1,
  Polynomial_regression_SG2     = reg_poly,
  Spline_regression_SG_bs4      = reg_spline,
  GAM_pos_on_SG                 = reg_gam,
  HMM_2_state_SG_putt           = hmm2_fit,
  HMM_3_state_SG_putt           = hmm3_fit
)
```

```
aic_core <- data.frame(
  Model = names(models_core),
  AIC   = sapply(models_core, safe_AIC)
) %>% dplyr::arrange(AIC)
```

```
aic_core
```

```
##                                     Model      AIC
## ARIMAX_sg_diff1_with_sg_total ARIMAX_sg_diff1_with_sg_total 140.4346
## HMM_2_state_SG_putt           HMM_2_state_SG_putt          198.4277
## HMM_3_state_SG_putt           HMM_3_state_SG_putt          202.0922
## GAM_pos_on_SG                 GAM_pos_on_SG                563.4867
## Polynomial_regression_SG2      Polynomial_regression_SG2     567.0524
## Linear_regression_pos_on_SG    Linear_regression_pos_on_SG   568.6156
## Spline_regression_SG_bs4       Spline_regression_SG_bs4     575.4537
## VAR_pos_and_SG                VAR_pos_and_SG               1186.9177
```